

Часть 1

Повторение и введение

Вычислительные задачи:

- 1) Задача распознавания языка — по слову x нужно выяснить, лежит ли оно в языке $L \subseteq \Sigma^*$. Алгоритм должен вернуть 1 на $\forall x \in L$ и 0 на $\forall x \notin L$.
- 2) Задача вычисления функции — по слову x надо найти $f(x)$, где $f: \Sigma^* \rightarrow \Sigma^*$. Как правило, считаем, что f всюду определена.
- 3) Задача поиска — по слову x нужно найти слово y такое, что xRy , где $R \subseteq \Sigma^* \times \Sigma^*$ — некоторое отношение. То есть на $\forall x \in \Sigma^*$ алгоритм должен или вернуть y такое, что xRy , либо сообщить, что таких y нет.

Вычислительные ресурсы:

- 1) Время работы. Так алгоритмы выполняются пошагово, то время будем мерить в шагах.
- 2) Память. Алгоритму необходимо пространство для промежуточных вычислений. Обычно в это пространство не включают вход программы.
- 3) Длина программы. Бывают ситуации, когда программа с более длинным текстом будет работать быстрее. Изучению этого фактора посвящена теория колмогоровской сложности.

Дополнительные инструменты:

- 1) Подсказка. Если машина получает небольшую подсказку, зависящую только от длины входа, то она может существенно ускориться. Более того, с подсказкой можно решить даже некоторые невычислимые функции. Использование подсказки моделирует ситуацию, в которой длина входа заранее известна.
- 2) Случайность. Некоторые задачи при современном уровне знаний решаются гораздо быстрее, если применить вероятностный алгоритм вместо детерминированного. Обычно платой за это ускорение будет маленькая, но положительная вероятность ошибки.
- 3) Интерактивность. Часто проверка результата может быть значительно проще его получения, поэтому можно использовать делегирование вычислений.
- 4) Специальные модели. Недетерминированные, альтернирующие и квантовые вычисления.
- 5) Оракулы. Вычисления с оракулом позволяют про любое слово за один шаг узнавать, принадлежит ли оно фиксированному множеству. Это отличается от вычислений с подсказкой тем, что на разных словах запросы к оракулу могут быть разными.

Асимптотические обозначения:

Пусть даны функции f и g такие, что $f: \mathbb{N} \rightarrow \mathbb{N}$; $g: \mathbb{N} \rightarrow \mathbb{N}$

- 1) $f(n) = O(g(n))$, если $\exists c \in \mathbb{R} \exists N \in \mathbb{N}: \forall n > N \hookrightarrow f(n) < cg(n)$
- 2) $f(n) = \Omega(g(n))$, если $\exists c > 0 \exists N \in \mathbb{N}: \forall n > N \hookrightarrow f(n) \geq cg(n)$
- 3) $f(n) = \Theta(g(n))$, если $\exists c \in \mathbb{R} \exists c > 0 \exists N \in \mathbb{N}: \forall n > N \hookrightarrow cg(n) < f(n) < Cg(n)$

- 4) $f(n) = o(g(n))$, если $\forall c > 0 \exists N \in \mathbb{N}: \forall n > N \Rightarrow f(n) < c g(n)$.
- 5) $f(n) = \omega(g(n))$, если $\forall c > 0 \exists N \in \mathbb{N}: \forall n > N \Rightarrow f(n) > c g(n)$.
- 6) $f(n) = \text{poly}(g(n))$, если $\exists c \in \mathbb{N} \cup \{0\} \Rightarrow f(n) = O((g(n))^c)$.
- 7) $f(n) = \text{polylog}(g(n))$, если $\exists c \in \mathbb{N} \cup \{0\} \Rightarrow f(n) = O((\log(g(n)))^c)$.
- 8) $f(n) = \tilde{O}(g(n))$, если $f(n) = O(g(n) \text{polylog}(g(n)))$.

Опр Машина Тьюринга - набор $(A; Q; Q_f; \delta; q_0; \lambda)$, где:

- A - конечное множество, называемое алфавитом МТ
- Q - конечное множество, называемое множеством состояний МТ, $A \cap Q = \emptyset$
- Q_f - множество финальных состояний МТ: $Q_f \subseteq Q$
- δ - функция (таблица) переходов: $\delta: A \times Q \rightarrow A \times \{-1; 0; 1\} \times Q$
- q_0 - начальное состояние: $q_0 \in Q$
- λ - пустой символ: $\lambda \in A$

Варианты МТ:

- Детерминированные (ДМТ) и недетерминированные (НМТ)
- Количество лент: 1 или больше
- Ленты, бесконечные в одну или в обе стороны
- Длина алфавита

В курсе используем k -ленточные ДМТ и НМТ. Количество лент k выбираем из удобства, но для каждого алгоритма оно фиксированно и не зависит от входа.

Часть 2

Распознавание языков на DMT и NMT. Классы DTIME и NTIME. Время работы

Распознавание языков на DMT

Опр Задача распознавания языка - по слову x нужно определить, лежит ли оно в языке $L \subseteq \Sigma^*$. Алгоритм должен вернуть 1 на $\forall x \in L$ и 0 на $\forall x \notin L$.

Опр МТ M распознаёт язык A за время $T(n)$, если она принимает все слова, лежащие в A , отвергает все слова, не лежащие в A и на $\forall x \in \Sigma^*$ работает $\leq T(|x|)$ шагов.

Здесь измеряется сложность в худшем случае: нужно, чтобы максимальное время работы на словах длины n не превысило $T(n)$.

Зам Если говорят "язык распознаётся за...", то без дополнительных оговорок имеется в виду многоленточная DMT с лентами, бесконечными в обе стороны.

Опр Классом $DTIME(T(n))$ называется класс языков, которые распознаются за время $O(T(n))$.

Зам Обычно в качестве $T(n)$ рассматривают только "хорошие" функции, т.е. такие $T(n) \geq n$, что $T(n)$ монотонна и её "можно быстро посчитать".

Опр Функция $T: \mathbb{N} \rightarrow \mathbb{N}$ называется конструируемой по времени, если $\exists$ алгоритм, который за время $O(\pi(n))$ по 1^n (т.е. числу n в унарной записи) получает $T(n)$ в бинарной записи.

Время работы:

- В прошлом семестре занимались тем, что узнавали, можем ли мы гарантировать разрешимость за конечное время.
 - В текущем семестре будем исследовать задачи на разрешимость за полиномиальное время на ДМТ и НМТ.
 - В текущем семестре также будем выяснять асимптотики конкретных алгоритмов и/или создавать алгоритмы с данной асимптотикой (как на курсе алгоритмов)
-

Детали модели:

Теорема (о минимуме ускорения, δ/g)

Пусть язык L разрешается машиной M , которая работает за время T . Тогда $\forall c > 0$
 $\exists$ МТ M' , эквивалентная M и работающая за время T' , где $T' = cT(n) + n + 100$

Теорема (малый алфавит, δ/g)

Пусть ДМТ M работает за время T и имеет алфавит A . Пусть входной и выходной алфавиты M равны $\{0, 1\}$. Тогда $\exists$ ДМТ M' , эквивалентная M , с алфавитом $\{t, 0, 1, \lambda\}$, которая работает за время $O(\log |A| \cdot T)$

Теорема (переход от многоленточной МТ к одноленточной, δ/g)

$\forall$ язык L , который можно распознать за время $O(T(n))$ на многоленточной МТ, можно распознать за время ~~$O(T^2(n))$~~ $O(T^2(n))$ на одноленточной МТ.

Теорема (про односторонние и двусторонние ленты, δ/g)

Лента, бесконечная в обе стороны, моделируется на односторонней ленте при помощи расширения алфавита. Если речь о многоленточной МТ, то лента, бесконечная в обе стороны, тривиально моделируется на двух односторонних лентах. Моделирование линейно по $T(n)$.

За δ прошлого семестра нам известно о существовании универсальной МТ U . Т.е. $\forall$ МТ M и $\forall$ слов $x; y \in \Sigma^*$: $M(x) = y \iff U(M; x) = y$. Теперь нас интересует время работы, поэтому заранее договоримся, что рассматриваем именно ДМТ.

Теорема (δ/g)

$\exists$ универсальная одноленточная ДМТ U , такая, что если M работает за $O(T(n))$, то U работает при моделировании M за $O(T^2(n))$.

Теорема (δ/g)

$\exists U$ -универсальная ДМТ: $U(M; x) = M(x)$ такая, что если M работает за время $T(n)$, то U при моделировании M работает за время $cT(n) \log(T(n)) + O(1)$, где $n = |x|$ и c зависит только от M .

Распознавание языков на НМТ

Рассмотрим НМТ. Единственным её отличием является неоднозначность переходов δ .

Опр НМТ M распознаёт язык L за время $T(n)$, если $\forall x \in \{0, 1\}^*$ верно:

1) $\nexists$ вычисления, начинающиеся с x , не длиннее $T(|x|)$, т.е. M в любом случае

остановится за не более чем $T(|x|)$ шагов.

2) Если $x \in L$, то $\exists$ вычисление, начинающееся с x и заканчивающееся в состоянии

q_{yes} .

3) Если $x \notin L$, то $\forall$ вычисление, начинающееся с x , заканчивается в состоянии

q_{no} .

Опр. Классом $NTIME(T(n))$ называется множество языков, распознаваемых на НМТ за
времени $O(T(n))$

Часть 3

Классы P, NP; со-классы. Сводимости по Карпу и Куку

Опр Классом P называется множество языков, распознающихся за полиномиальное время, или же:

$$P = \bigcup_{c=1}^{\infty} O(\text{TIME}(n^c)) = O(\text{TIME}(\text{poly}(n)))$$

Примеры языков из P:

1) $\text{GCD} = \{(a, b, d) \mid \text{НОД}(a, b) = d\}$

2) $\text{PATH} = \{(G, s, t) \mid \text{в графе } G \text{ есть путь из } s \text{ в } t\}$

3) $\text{EULER CYCLE} = \{G \mid \text{в графе } G \text{ есть эйлеров цикл}\}$

4) $\text{BIPARTITE} = \{G \mid \text{граф } G \text{ двудольный}\}$

Опр (через NTIME) Классом NP называется множество языков, распознающихся на НМТ за полиномиальное время, или же:

$$NP = \bigcup_{c=1}^{\infty} N\text{TIME}(n^c) = N\text{TIME}(\text{poly}(n))$$

Опр (сертификатное) Классом NP называется множество языков L , для которых

$\exists$ функция $V: \Sigma^* \times \Sigma^* \rightarrow \{0, 1\}$ такая, что $V(x, s)$ ~~вычисляется~~ вычислима за полиномиальное время от длины первого аргумента. и выполняется:

1) $x \in L \Leftrightarrow \exists s: V(x, s) = 1$

2) $x \notin L \Leftrightarrow \forall s: V(x, s) = 0$

Второй вход s часто называют сертификатом, а функцию V — верификатором.

Таким образом, сертификат удостоверяет, что $x \in L$, а верификатор проверяет верность

сертификата.

Теорема Определение NP через $NTIME \Leftrightarrow$ сертификатное определение NP

□ $\Rightarrow$ Пусть язык L распознаётся за полиномиальное время на НМТ. Построим верификатор и сертификат. Пусть мы будем в каждой ветви вычислений НМТ сохранять последовательность выборов переходов. Такая последовательность переходов и будет сертификатом. НМТ V будет моделировать работу исходной НМТ по сертификату, то есть, зная переходы однозначно. Т.к. каждая ветвь вычислений ~~работает~~ за не длинее некоторого полинома $p(|x|)$, то V будет работать $O(p(|x|))$ шагов. При этом если $\exists$ ветвь вычислений, приводящая к 1, то НМТ принимает слово. Но тогда и $\exists$ сертификат s , равный этой последовательности выборов переходов такой, что $V(x, s) = 1$ и V работает за полином. Если же такой ветви нет, то на $\forall$ сертификате s будет $V(x, s) = 0$ и V будет работать за полином $\Rightarrow$ доказано, что $NP \subseteq NP_c$

$\Leftarrow$ Пусть нам дан верификатор V , который на $\forall x$ работает $\leq \Sigma^*$ и $\forall s$ работает за $\leq p(|x|)$ шагов. Тогда мы можем сказать, что $|s| \leq p(|x|)$, иначе V не сможет считать сертификат полностью. То есть мы можем наложить ограничение сверху на длину сертификата. Создадим НМТ M , которая делает следующее: перебирает все сертификаты длины до $p(|x|)$. НМТ это может сделать за $O(p(|x|))$, задав для каждого слова свою ветвь вычислений. Далее в каждой ветви она запускает $V(x, s)$, который тоже работает за $O(p(|x|))$. Тогда если $\exists s: V(x, s) = 1$, то НМТ в какой-то

ветви его переберёт и, следовательно, примет слово. Если же сертификата нет, то во всех ветвях будет 0 $\Rightarrow$ НМТ отклонит слово. $\Rightarrow NP_2 \subseteq NP_1 \Rightarrow$

$\Rightarrow$ определения эквивалентны ■

Примеры языков из NP:

- 1) Все задачи из класса P
- 2) $SAT = \{ \varphi \mid \varphi - \text{выполнимая булева формула} \}$
- 3) $CLIQUE = \{ (G, k) \mid \text{в графе } G \text{ найдётся полный подграф хотя бы на } k \text{ вершинах} \}$
- 4) $3COL = \{ G \mid \text{все вершины графа } G \text{ можно правильно раскрасить в 3 цвета} \}$
- 5) $MAXCUT = \{ (G, k) \mid \text{все вершины графа } G \text{ можно разбить на 2 непересекающихся множества } S \text{ и } T \text{ так, что между частями проведено не менее } k \text{ рёбер} \}$

Гипотеза $P \neq NP$

Читателю предлагается попробовать доказать это самостоятельно :)

Опр Для класса C класс coC определяется так: $L \in coC \Leftrightarrow \bar{L} \in C$

Опр $L \in coNP \Leftrightarrow \bar{L} \in NP$.

Для $coNP$ можно сформулировать сертификатное определение по аналогии с NP:

Опр $L \in coNP \Leftrightarrow \exists V(x; s)$ - вычислимая за $O(\text{poly}(|x|))$ функция с булевыми значениями

такая, что:

1) $x \in L \Leftrightarrow \forall s V(x; s) = 1$

2) $x \notin L \Leftrightarrow \exists s V(x; s) = 0$

$$\text{УТВ } P \subseteq NP \cap coNP$$

□ W ■

Гипотеза $NP \neq coNP$

Опр Пусть A и B - два языка. Тогда A сводится по Карпу к B , если $\exists$ всегда определённая функция $f: \{0,1\}^* \rightarrow \{0,1\}^*$, вычисляемая за полиномиальное время, такая, что:

$$x \in A \Leftrightarrow f(x) \in B$$

Обозначение: $A \leq_P B$

Свойства сводимости по Карпу:

- 1) $A \leq_P A$ - рефлексивность
- 2) $A \leq_P B; B \leq_P C \Rightarrow A \leq_P C$ - транзитивность
- 3) $A \in P; B \neq \emptyset; B \neq \{0,1\}^* \Rightarrow A \leq_P B$
- 4) $B \in P; A \leq_P B \Rightarrow A \in P$
- 5) $B \in NP; A \leq_P B \Rightarrow A \in NP$

Опр Сведение задачи R_1 к R_2 по Куку - это полиномиальный по времени алгоритм (ОМТ с полиномиальным временем работы), решающий задачу R_1 при условии, что функция, находящая решение R_2 , дана ему в качестве оракула, то есть обращение к ней занимает всего один шаг.

Обозначение: $R_1 \leq_C R_2$

Часть 4

NP-трудность и NP-полнота. Существование NP-полного языка. Теорема Кука-Левина

Опр Язык B является NP-трудным, если $\forall A \in NP \cup A \leq_p B$

Опр Язык B является NP-полным, если $B \in NP$ и B - NP-трудный.

Утв $B \in P$; B - NP-трудный $\Rightarrow P = NP$

Утв A - NP-трудный; $A \leq_p B \Rightarrow B$ - NP-трудный.

Зам Множество NP-полных языков будем обозначать NPC .

Теорема $\exists L_{NP} = \{ \langle N, x, 1^t \rangle \mid \text{НМТ } N \text{ принимает } x \text{ за время } t \}$. Тогда

$L_{NP} \in NPC$

$\square$ Выберем произвольный $L \in NP$. Он распознаётся какой-то НМТ N_L за время

$p(|x|)$, где $p(|x|) = O(\text{poly}(|x|))$. Тогда, выписывая за полином функции S делаем следующее: $S_{L(x)} = (N_L; x, 1^{p(|x|)})$. Из построения следует, что

$x \in L \Leftrightarrow S(x) \in L_{NP}$. Таким образом, $\forall L \in NP \cup L \leq_p L_{NP} \Rightarrow L_{NP}$ - NP-трудный.

$L_{NP} \in NP$, т.к. можно смоделировать работу N на x в течение t шагов, и это будет полиномиально относительно длины входа. $\Rightarrow L_{NP} \in NPC$ ■

Теорема (Кука-Левина)

$SAT = \{ \varphi \mid \varphi - \text{выполнимая булева формула} \}$. $SAT \in NPC$.

$\square$ 1) Сначала докажем, что $SAT \in NP$. Для этого надо придумать верификатор и сертификат. В качестве сертификата будет набор из значений для каждой переменной.

ной. Т.к. каждая переменная встречается в формуле хотя бы один раз, то количество переменных $n \leq |\varphi|$. $\Rightarrow$ длина сертификата полиномиальна относительно длины входа. Верификатор же будет брать этот фиксированный набор значений, подставлять в формулу и вычислять. Это тоже работает за полином от $|\varphi| \Rightarrow$
 $\Rightarrow \text{SAT} \in \text{NP}$

2) Теперь докажем, что SAT - NP-трудный.

Выберем произвольный $L \in \text{NP}$. Хотим доказать, что $L \leq_p \text{SAT}$. То есть построить f - полиномиально вычисляемую от $|x|$ такую, что $x \in L \Leftrightarrow f(x) \in \text{SAT}$

Т.к. $L \in \text{NP}$, то $\exists$ верификатор V . Тогда p -сводимость будет заключаться в том, что мы по слову x , зная верификатор V , хотим построить булеву формулу φ такую, что φ -выполнима $\Leftrightarrow \exists$ сертификат s для верификатора V . Также для языка L есть некий полином $p(|x|)$, который ограничивает сверху время работы верификатора.

Давайте сделаем верификатор специального вида (такой всегда $\exists$):

- 1) Верификатор - однокленточная ДМТ с лентой, бесконечной в одну сторону.
- 2) Алфавит верификатора $A = \{ \blacktriangleright; \#; 0; 1; \blacktriangleleft; \wedge \}$.
- 3) Входная лента всегда выглядит так:

$\begin{array}{ccccccc} & |x| & & q(|x|) & & & \\ \blacktriangleright & 0 & 1 & 0 & \# & 1 & 0 & \# & \wedge & \dots & \wedge & \blacktriangleleft \\ & \underbrace{\hspace{2cm}}_{\text{бинарный код } x} & & \underbrace{\hspace{2cm}}_{\text{сертификат } s} & & & & & & & \end{array}$

Затем мы так сделали? Лента, бесконечная в одну сторону, гарантирует, что мы не выйдем левее символа $\blacktriangleright$. Также в качестве входа мы подали верификатору

x и s . $|s| = q(|x|)$, то есть длина сертификата фиксированна. Такое
 отражение можно дополнительно наложить, чтобы было верное определение NP
 (см. задачу 7 из 8/3). Далее мы заполним ленту символами так, чтобы
 всего на ленте было $p(|x|)$ символов. Т.к. верификатор работает $\leq p(|x|)$ шагов,
 то он никогда не сможет выйти правее символа $\blacktriangleleft$. Таким образом $\nexists$
 конфигурация верификатора V может быть представлена как набор символов
 только на указанных ячейках (т.е. $p(|x|)$ ячеек) и перед текущим символом, есть
 один символ состояния. $\Rightarrow$ длина корректной конфигурации $V \leq p(|x|) + 1$,
 далее справа на ленте всегда только $\perp$.

Теперь возьмём все символы из $A \cup Q$ - алфавит и множество состояний V
 соответственно. Пусть $|A \cup Q| = N$ - константа, не зависящая от x . Тогда для
 кодировки всех символов из $A \cup Q$ хватит $m = \lceil \log_2 N \rceil$ бит, то есть вместо
 каждого символа мы будем писать его двоичный код длины m .

Построим таблицу размером $p(|x|) \times m(p(|x|) + 1)$, т.е.

$x_{11} x_{12} \dots x_{1m}$	$x_{1m+1} x_{1m+2} \dots x_{12m}$	...	$x_{1m(p(x)+1)} \dots x_{1m(p(x)+1)m}$
$x_{21} x_{22} \dots x_{2m}$			
$\vdots$			
$\vdots$			
$\vdots$			
$x_{p(x)+1} \dots x_{p(x)+1m}$			

Пусть каждая ячейка - уникальная переменная из булевой формулы $\Rightarrow$ будет $p(x_i) = m(p(x_i)+1)$ переменных. Хотим как-то построить формулу, но она будет выполнима $\Leftrightarrow$ каждая строка таблицы - корректная конфигурация V , все $p(x_i)$ строк образуют корректный протокол V и V принимает x .

Назвём создавать эту булеву формулу. Сначала проверим корректность первой строки. Для этого мы смо провераем, что первые m символов - корректное двоичное кодирование символа $\blacktriangleright$, потом аналогично проверяем на корректность все символы $\chi; \#; \wedge$ и $\blacktriangleleft$ и из первой строки мы пропускаем только сертификат, ведь мы это не знаем. Для понимания, как это будет выглядеть в формуле. Пусть $m=5$;

$\blacktriangleright = 10011$. Тогда проверка, что первые 5 символов задают $\blacktriangleright$ будет выглядеть, как $(x_{11} \equiv 1) \wedge (x_{12} \equiv 0) \wedge (x_{13} \equiv 0) \wedge (x_{14} \equiv 1) \wedge (x_{15} \equiv 1)$. Проверка остальных символов делается

аналогично. Все разделители между проверками всех ячеек - конъюнкции. Так мы проверили корректность первой конфигурации. Научимся проверять корректность всех остальных.

Рассмотрим ζ подряд идущих ~~ячеек~~ ^{блока из m клеток} в строке и один ~~символ~~ ^(т.е. 4 символа исходно) ниже, как показано далее:



Утверждается, что по верхним ζ ^{блокам} ~~символам~~ можно гарантированно проверить этот ~~символ~~ ^{блок} снизу. Рассмотрим все ситуации. Далее ~~а, в, с, d, q~~ a, b, c, d, q

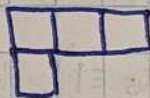
$a, b, c, d, a', b', c', d' \in A; q, q' \in Q$. За y обозначим значение в блоке снизу.

Тогда: $abcd \Rightarrow y \equiv b$; $qbcd \Rightarrow \begin{cases} y \equiv b'; \text{ если } \delta(q, b) = (q', -1; b') \\ y \equiv q'; \text{ если } \delta(q, b) = (q', 1; b') \end{cases}$

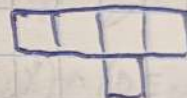
$$\begin{aligned}
 aqc \Rightarrow & \begin{cases} y \equiv a, & \text{если } \delta(q, c) = (q', -1; c') \\ y \equiv q', & \text{если } \delta(q, c) = (q', 0; c') \\ y \equiv c', & \text{если } \delta(q, c) = (q', 1; c') \end{cases} ; \quad acqb \Rightarrow \begin{cases} y \equiv q', & \text{если } \delta(q, a) = (q', -1; b') \\ y \equiv c, & \text{если } \delta(q, a) = (q', 0; b') \end{cases}
 \end{aligned}$$

$$absc \Rightarrow y \equiv b$$

Все такие проверки делаются побитовым сравнением, которое если оформляется через импликацию, то условие $\rightarrow$ проверка при этом условии. Для каждого блока надо проверить N^4 значений верхних блоков и далее расписать проверки, то есть каждая формула для каждого блока будет длины $O(1)$, так не зависит от n , а зависит только от V . Так, мы можем проверить корректность всех символов, кроме самого левого в строке и двух самых правых. Для них можно провести аналогичные рассуждения, рассмотрев:



- для левого;



- для

края самого правого;  - для правого.

Осталось проверить, что V принял слово. Для этого достаточно проверить, что один из блоков в нижней строке - код q_{yes} - принимающего блока. То есть проверяем, что хотя бы один из блоков - q_{yes} - побитовое сравнение с конъюнкцией для проверки символов внутри блока и дизъюнкцией между проверками блоков.

Итак, за $O(p(n) \cdot t(p(n) + 1)) = O(p(n) \cdot t(n))$ построим формулу φ такую, что

$$\varphi \text{ - выполнима} \Leftrightarrow x \in L \Rightarrow L \leq_p SAT \quad \forall L \in NP \Rightarrow SAT \text{ - NP-трудный} \Rightarrow$$

$$\Rightarrow SAT \in NPC \blacksquare$$

Теорема

SAT

Тогда и

Теорема (8/9) Пусть $3SAT = \{\varphi \mid \varphi - \text{выполнимая булева формула в виде 3КНФ}\}$.

$$SAT \leq_p 3SAT$$

Тогда из этой теоремы следует, что $3SAT \in NPC$

Часть 5

Примеры NP-полных задач

① $INDSET = \{(G, k) \mid \text{в графе } G \text{ есть независимое множество из } k \text{ вершин}\}$

Напоминание: независимое множество означает, что между вершинами этого множества нет ребер.

Теорема $INDSET \in NPC$

□ 1) Докажем, что $INDSET \in NP$. Заметим, что в качестве сертификата можно взять набор из k вершин. Верификатор будет перебирать все пары вершин из множества и проверять на наличие в графе ребра между ними. Этот алгоритм работает за $O(n^2)$, т.е. верификатор полиномиально работает от длины входа. Помимо этого $x \in INDSET \Leftrightarrow \exists$ сертификат, т.к. сертификат и есть независимое множество из k вершин.

2) Докажем, что $INDSET$ - NP-трудный. Хотим $3SAT \leq_p INDSET$. Это для этого надо сделать. Придумать полиномиально вычисляемую функцию f , которая по формуле строит граф и число так, что формула выполнима $\Leftrightarrow$ в графе есть независимое множество требуемого размера. Пусть дана функция φ , f строит граф G и число k . Общий алгоритм для f :

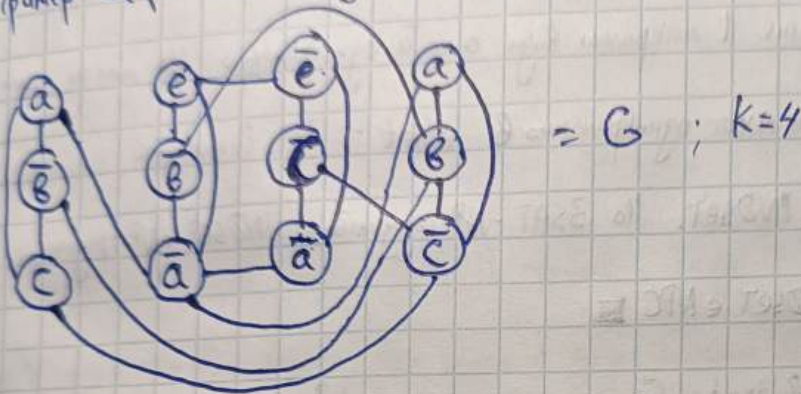
1) Проходимся по всем дизъюнктам в ЗКНФ. Для каждого дизъюнкта добавляем в изначально пустой G 3 новые вершины. Каждая из новых вершин соответствует литералу в дизъюнкте. Пусть вершина заполнит литерал, которому она соответствует. Все 3 новые вершины соединим попарно друг с другом неориентированными ребрами.

тированными рёбрами.

3) Если вершина соответствует некой переменной x , то соединим её неориентированными рёбрами со всеми вершинами, которые соответствуют $\bar{x}$.

4) Выберем k , равное количеству дизъюнктов.

Пример $f(\varphi) = (G, k)$, где $\varphi = (a \vee \bar{b} \vee c) \wedge (e \vee \bar{b} \vee \bar{a}) \wedge (\bar{e} \vee c \vee a) \wedge (a \vee b \vee \bar{c})$:



Заметим, что f вычисляется за полином от длины формулы (за квадрат).

Осталось доказать, что φ -выполнима $\Leftrightarrow$ в G есть indset размера k

$\Rightarrow$ Пусть φ -выполнима. Тогда значение каждого дизъюнкта $= 1$ на выбранном наборе выполняющих наборе. Тогда в каждом дизъюнкте есть хотя бы один литерал со значением 1, если таких несколько, то выберем любой. Теперь рассмотрим множество, состоящее из вершин, соответствующих выбранным литералам. Оно размера k . Докажем, что в нём нет рёбер. Рёбра бывают двух видов:

1) Между вершинами из одного дизъюнкта. Таких во множестве нет, т.к. все из разных дизъюнктов

2) Между литералами вида $a \rightarrow \bar{a}$. Таких тоже нет, т.к. они всегда имеют разные

значения, а мы выбрали только те, которые равны 1.

Таким образом, ребро нет $\Rightarrow$ нашли indset размера k

⑤ Рассмотрим независимое множество размера k . Заметим, что все вершины из разных дизъюнктов. Тогда, если все эти литералы принимают значение 1, то в каждом дизъюнкте φ есть 1 $\Rightarrow$ нашли выполняющий набор. Противоречия нам бы быть только если присвоили значение 1 литералам вида a и $\bar{a}$ одновременно. Но между ними есть ребро, поэтому они не лежат одновременно в $\text{indset} \Rightarrow \varphi$ - выполнима.

Таким образом $3\text{SAT} \leq_p \text{INDSET}$. Но 3SAT - NP-трудный $\Rightarrow \text{INDSET}$ - NP-трудный.

Т.к. $\text{INDSET} \in \text{NP} \Rightarrow \text{INDSET} \in \text{NPC}$ ■

② $\text{CLIQUE} = \{G, k \mid \text{в графе } G \text{ есть клика размера } k\}$

Теорема $\text{CLIQUE} \in \text{NPC}$

1) Докажем, что $\text{CLIQUE} \in \text{NP}$. Заметим, что можно взять в качестве сертификата набор из k вершин, а верификатор будет проверять, что между любыми двумя есть ребро. Аналогично, как для INDSET .

2) Докажем, что CLIQUE - NP-трудный. В целом, достаточно показать, что если в графе G есть независимое множество ^{размера k} , то в графе G' (все ребра G отсутствуют в G' , все отсутствующие в G ребра есть в G') есть клика размера k на тех же вершинах, что были в indset . Тогда пусть $f(G, k) = (G', k)$ и получаем, что в G есть indset размера $k \Leftrightarrow$ в G' есть клика размера k .

f вычислима за $O(|V|^2)$, т.е. за полином от длины входа. $\Rightarrow$

$\Rightarrow \text{INDSET} \leq_p \text{CLIQUE}$. Т.к. $\text{INDSET} \in \text{NPC}$, то CLIQUE - NP-трудный $\Rightarrow \text{CLIQUE} \in \text{NPC}$ ■

■

③ $\text{VERTEXCOVER} = \{ \langle G, k \rangle \mid \text{в графе } G \text{ есть вершинное покрытие размера } k \}$

Напоминание: Набор вершин называется вершинным покрытием, если $\nexists$ ребра такого, что оба его конца не лежат в вершинном покрытии.

Теорема $\text{VERTEXCOVER} \in \text{NPC}$

□ 1) $\text{VERTEXCOVER} \in \text{NP}$ эквивалентно INDSET и CLIQUE .

2) Заметим следующий факт: Пусть S - indset . Тогда $V \setminus S$ - вершинное покрытие, где V - множество всех вершин. Нет ребра с двумя концами в $S \Rightarrow \forall$ ребра есть

вершина из $V \setminus S \Rightarrow V \setminus S$ - вершинное покрытие. Обратное аналогично доказывается.

Тогда $f(G, k) = (G, |V| - k)$ - за вычисления за $O(1)$. $\Rightarrow \text{INDSET} \leq_p \text{VERTEXCOVER} \Rightarrow$

$\Rightarrow \text{VERTEXCOVER}$ - NP-трудный $\Rightarrow \text{VERTEXCOVER} \in \text{NPC}$ ■

④ $\text{3COL} = \{ \langle G \rangle \mid \text{вершины графа } G \text{ можно правильно раскрасить в 3 цвета} \}$

Напоминание: правильная раскраска означает, что в графе нет рёбер между вершинами одного цвета.

Теорема $\text{3COL} \in \text{NPC}$

1) Докажем, что $\text{3COL} \in \text{NP}$. В качестве сертификата будет раскраска, т.е. для каждой вершины графа указан её цвет. Верификатор же будет проходить по всем рёбрам графа и проверять, разноли цвета концы рёбер.

2) Хотим $3SAT \leq 3COL$. То есть по формуле надо построить граф. Приведем общий алгоритм построения графа G по формуле φ (т.е. высказывания $\mathcal{L}$).

Для начала скажем, что вершины мы будем красить в 3 цвета: T -true, F -false, N -null. Идеа в том, что все литералы смогут иметь только цвета T и F , и в графе была раскраска в 3 цвета $\Leftrightarrow \varphi$ -выполнима $\Leftrightarrow$ в каждом дизъюнкте есть вершина цвета T . Итак, алгоритм построения графа:

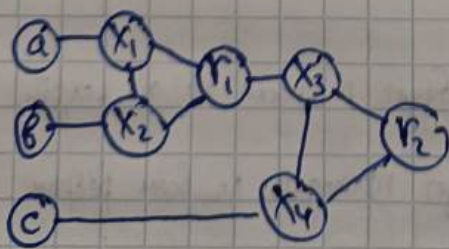
1) Зададим 3 стартовые вершины и соединим их все попарно ребрами. Все вершины будут разного цвета. Назовем вершины, соответствующие цветам T, F и $N \rightarrow T_0, F_0$ и N_0 соответственно.

2) Зададим вершины для всех $3d$ литералов из φ (где d - число дизъюнктов)

3) Соединим все литеральные вершины с N_0 (теперь они не могут быть цвета N).

4) Если две вершины соответствуют литералам a и $\bar{a}$, то соединим их ребром (теперь они обязаны быть разноцветными, т.е. принимать разные значения).

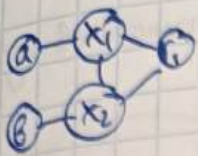
5) Пусть есть дизъюнкт $(a \vee b \vee c)$, где a, b, c - литералы. Тогда выберем вершины, соответствующие этим литералам. (На рисунке $\textcircled{a}, \textcircled{b}, \textcircled{c}$). Построим в графе следующую конструкцию:



Здесь вершины $x_1, x_2, x_3, x_4, r_1, r_2$ - новые в графе. Также соединим r_2 с F_0 и N_0 , чтоб она была цвета T .

Докажем, что если $a=b=c=\text{false}$, то раскрасить такую конструкцию невозможно, а

иначе возможно. Сначала рассмотрим кусок:



Заметим, что если $a=b$, то $r_1=a=b$. Это связано с тем, что вершины x_1, x_2, c разного цвета, но x_1 и x_2 не могут быть

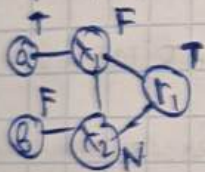
того же цвета, что и a и b . $\Rightarrow$ если $a=b=0$, то $r_1=0$, если $a=b=1$, то $r_1=1$.

Заметим, что вершины c, r_1, x_2, x_1, r_2 образуют аналогичную конструкцию, и если $a=b=c=0$, то $r_1=0 \Rightarrow r_2=0$, но r_2 должно быть цвета $T \Rightarrow$ если φ -невыполнима, то

раскрасить невозможно. Теперь покажем, что если $a \neq b=1$, то можно положить $r_1=1$.

Из этого будет следовать, что r_2 может быть цвета T . $\Rightarrow$ мы всё успешно раскрасим.

Примеры раскрасок для всех случаев (для $a=b=1$ уже доказано, что $r_1=1$, а 2 других случая симметричны):



- успешно покрасили.

Итого, мы потратили за полиномиальное время строить граф. И если формула φ выполнима, то граф 3-раскрашиваемый (все литералы цвета T или F , литералы вида a и $\bar{a}$ разного цвета; во всех дизъюнктах есть 1). Если φ выполнима, то покрасим литеральные вершины в цвета их значений, и раскраска не сломается $\Rightarrow$ φ -выполнима $\Rightarrow G$ - 3-раскрашиваемый.

Если же φ -невыполнима, то есть дизъюнкт, где все 0 ^{Вместо бора} $\Rightarrow$ по доказанному ранее мы не сможем покрасить ветку для такого дизъюнкта, как бы мы ни красили литералы $\Rightarrow$ 3-раскраски нет, т.е. φ -невыполнима $\Rightarrow G$ - не 3-раскрашиваемый.

Таким образом, φ -выполнима $\Leftrightarrow G$ - 3-раскрашиваемый $\Rightarrow 3SAT \leq_p 3COL \Rightarrow$

$\Rightarrow 3COL$ - NP-трудный $\Rightarrow 3COL \in NPC$ ■

⑤ $HAMPATH = \{ \langle G, s, t \rangle \mid \text{в ориентированном графе } G \text{ есть гамильтонов путь из вершины } s \text{ в вершину } t \}$

Напоминание: гамильтонов путь - путь проходящий по всем вершинам графа ровно по одному разу

Теорема: $HAMPATH \in NPC$

1) Докажем, что $HAMPATH \in NP$. В качестве сертификата выберем перестановку вершин. Верификатор будет проверять, что между соседями есть ребро.

2) Теперь докажем, что $HAMPATH$ - NP-трудный. Для этого хотим $3SAT \leq_p HAMPATH$.

То есть по формуле φ будем строить граф G и вершины s и t так, что

φ -выполнима $\Leftrightarrow$ в G есть гамильтонов путь из s в t . Приведём алгоритм построения G .

1) Пусть в ЗКНФ d дизъюнктов. Добавим в граф d вершин, соответствующих дизъюнктам. Назовём их $D_1, D_2, \dots, D_d$.

2) Пусть в формуле n переменных. Зададим для каждой переменной $d+3$ вершины

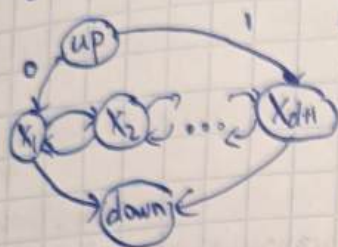
3) Добавим в G вершины s и t просто для того, чтоб было удобно определять начало и конец пути.

Итого мы создали $d + n(d+3) + 2$ вершины в G , больше добавлять не будем. Это

в худшем случае квадратично от длины входа, потому что формулы линейна от

гипотеза дизъюнктив, а $n \leq 3d$.

Теперь добавим в граф рёбра. Рассмотрим произвольную переменную x , ниже я обозначу все $d+3$ вершины, которые были для неё созданы:



$2d+9$ рёбра строим, как показано на рисунке.

Идея в том, что мы зайдём в вершину up , и у нас есть ровно 2 обхода всех вершин — по "цепи" налево или

направо. Зафиксируем, какое из направлений будет соответствовать тому, что $x=0$,

а какое — $x=1$. Далее мы хотим понять, какая вершина в каком случае может выполнять

какой-то дизъюнкт. Пусть в дизъюнкте φ_i есть литерал x . Тогда при $x=1$ этот дизъюнкт

выполняется, тогда мы хотим иметь возможность зайти в вершину φ_i дополнительно,

не нарушая обход. Аналогично, если встретим $\bar{x}$, то для направления 0 надо посетить

φ_i выполняем. Алгоритм такой:

Пусть литерал x встречается в φ_i . Тогда проводим рёбра $x_{in} \rightarrow \varphi_i$ и

$\varphi_i \rightarrow x_i$, если $\bar{x}$ встречается в φ_i , то проводим рёбра $x_i \rightarrow \varphi_i$, $\varphi_i \rightarrow x_{in}$.

~~Приведём пример графа для~~

Далее пронумеруем все переменные произвольным образом и назовём их $x_1, x_2, \dots, x_n$.

Проведём в графе рёбра: $s \rightarrow (x_1-up)$; $(x_1-down) \rightarrow (x_2-up) \dots (x_n-down) \rightarrow t$.

Теперь мы полностью построили граф. Внутри каждой переменной мы создали $2d+4$

рёбра, для связи между переменными, а s и t построили $n+1$ рёбро. Осталось

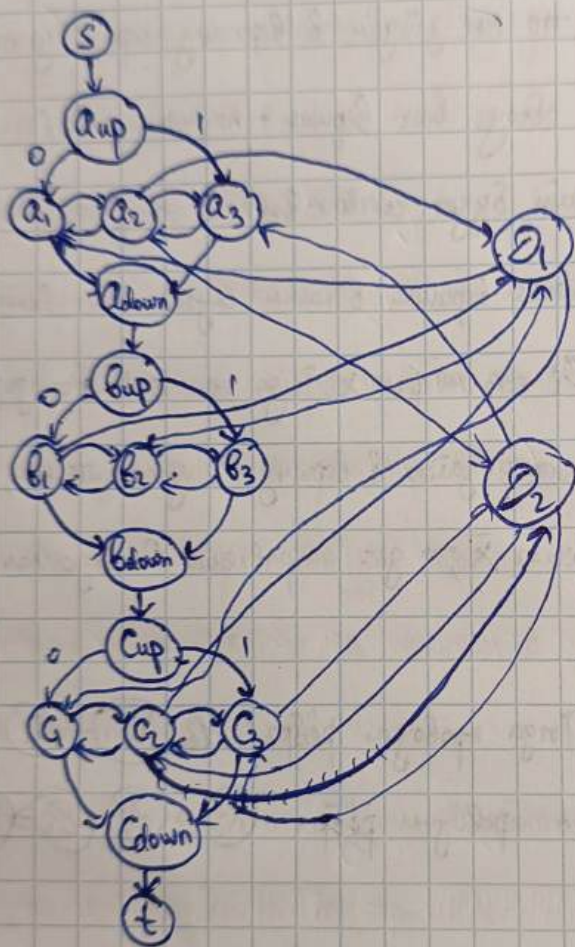
посчитать число рёбер, которые мы проводим к дизъюнктам. Всего $3d$

литералов, каждый литерал добавляет ровно 2 ребра. Итого рёбер в графе:

$(2n+4)n + n+1 + 6d$, что тоже квадратично от длины входа $\Rightarrow$ граф строится по φ

за полином; s и t мы тоже знаем.

Пример графа для $\varphi = (a \vee \bar{b} \vee c) \wedge (\bar{a} \vee \bar{c} \vee d)$:



Тяжело нормально нарисовать, но надеюсь, что понять можно.

Итак, теперь докажем утверждение:

φ - выполнима $\Leftrightarrow$ в G есть замкнутый путь из s в t

$\Rightarrow$ Выберем $\forall$ выполняющий набор. Тогда мы знаем направление обхода каждой переменной. И, что всегда можно обойти все вершины, которые не вида d_i , но

по пути нам надо вместо каких-то рёбер вида $(x_i) \rightarrow (x_{i+1})$ или $(x_{i+1}) \rightarrow (x_i)$

выбирать $(x_i \rightarrow x_{i+1} \rightarrow x_{i+2})$ или $(x_{i+1} \rightarrow x_i \rightarrow x_{i+2})$ соответственно, чтобы посетить все дизъюнктивные вершины. Тогда давайте заходить в x_i сразу, как только сможем, а мы всегда сможем, поскольку дизъюнкты выполняются.

⊖ Докажем, что если φ невыполнима, то нет гамильтонова пути. Каждый подграф, соответствующий переменной x_i мы обязаны пройти в одном из 2 направлений. Каждому такому направлению соответствует набор значений. Если для всех наборов результат 0, то мы не сможем по пути зайти в вершину, соответствующую дизъюнкту со значением 0 $\Rightarrow$ не прошли все вершины $\Rightarrow$ нет гамильтонова пути.

Таким образом, $3SAT \leq_P \text{HAMILTON}$ $\Rightarrow$ HAMILTON - NP-трудный $\Rightarrow$ $\text{HAMILTON} \in \text{NPC}$ ■

⊕ $\text{HAMILTON} = \{ \langle G, s, t \rangle \mid \text{в неориентированном графе } G \text{ есть гамильтонов путь из } s \text{ в } t \}$

Теорема $\text{HAMILTON} \in \text{NPC}$

□ 1) $\text{HAMILTON} \in \text{NP}$, аналогично HAMILTON

2) Хотим $\text{HAMILTON} \leq_P \text{HAMILTON}$. То есть по ориентированному графу G ; вершинам s и t построить неориентированный граф G' и выбрать для него источник и сток.

Для вершины s создадим 2 вершины в G' : s_{mid} и s_{out} ; для t тоже 2 вершины:

t_{in} и t_{mid} , для остальных 3 вершины in, mid и out . Для каждой старой вершины x проведем рёбра $x_{in} \leftrightarrow x_{mid}$; $x_{mid} \leftrightarrow x_{out}$, для s и t тоже, только у s и t по 1 ребру.

Далее будем перекосить рёбра из G . Если ориентированное ребро имеет конец в s или начало в t , то игнорируем его. Иначе, пусть есть ребро $a \rightarrow b$.

Теорема $TSP \in NPC$

□ 1) $TSP \in NP$, потому что в качестве сертификата можно взять перестановку вершин, а верификатор будет проверять, есть ли рёбра между соседями, считать длину пути и сравнивать с L

2) Хотим $HNATH \leq_p TSP$. Для этого всем вершинам исходного графа зададим вес 1, скажем $L = |V| - 1$. Тогда получим абсолютно эквивалентные задачи $\Rightarrow$

$\Rightarrow HNATH \leq_p TSP \Rightarrow TSP$ - NP-трудный $\Rightarrow TSP \in NPC$

⑧ $EXACTSETCOVER = \{ (n; S_1; S_2 \dots S_m) \mid \text{можно выбрать непересекающиеся множества}$

$$S_1, S_2 \dots S_k : \bigcup_{j=1}^k S_j = \{1; 2 \dots n\} \}$$

Комментарий: далее будем считать, что у нас не числа от 1 до n , а некоторое множество из n различных элементов (можно вообще просто на концы заменить), а также, что разных элементов во множествах не встречается.

Теорема $EXACTSETCOVER \in NPC$

□ 1) $EXACTSETCOVER \in NP$, потому что можно в качестве сертификата взять набор множеств, а верификатор будет проверять, что объединение множеств даёт все элементы, и что множества не пересекаются.

2) Хотим $3SAT \leq_p EXACTSETCOVER$. То есть по 3КНФ φ надо построить набор множеств.

Заведём для каждой переменной x 3 буквы: $x; x_0; x_1$ и для каждого дизъюнкта

φ 4 буквы: $\varphi; \varphi_1; \varphi_2; \varphi_3$ и получаем, как число заведённых букв. Теперь будем строить множества. Для каждой переменной x заведём множества $\{x; x_0\}$ и $\{x; x_1\}$,

а для каждого дизъюнкта $\mathcal{D}$ заводим множества $\{\mathcal{D}\}; \{\mathcal{D}, \mathcal{D}_1\}; \{\mathcal{D}, \mathcal{D}_2\}; \{\mathcal{D}, \mathcal{D}_3\};$
 $\{\mathcal{D}, \mathcal{D}_1, \mathcal{D}_2\}; \{\mathcal{D}, \mathcal{D}_1, \mathcal{D}_3\}; \{\mathcal{D}, \mathcal{D}_2, \mathcal{D}_3\}$. Также для каждой переменной x создадим 2
 множества вида: $\{x_0; A_1; B_1; C_2 \dots\}; \{x_1; D_2; E_3 \dots\}$. В первом множестве помимо
 x_0 перечислены все места литерала x в формуле (т.е. дизъюнкт + позиция в нём),
 аналогично строится второе множество, но для литерала $\bar{x}$. Нетрудно показать, что
 суммарная мощность множеств полиномиальна от длины ввода.

Итак, мы получили набор множеств, осталось доказать, что:

φ - выполнима $\Leftrightarrow \exists \text{ exact set cover}$ из таких множеств

$\Rightarrow$ Пусть есть выполняющий набор, возьмём его. Это мы тогда выберем в качестве
 покрытия? Сначала разберёмся с переменными. Пусть $x=1$. Тогда выберем множества

$\{x, x_1\}$ и $\{x_0, \dots\}$. Иначе берём $\{x, x_0\}$ и $\{x_1, \dots\}$. Тогда мы
 тут дизъюнкты тут дизъюнкты

покрыли все буквы, которые мы заводили для переменных, при этом все полученные
 множества не пересекаются. Осталось разобраться с буквами для дизъюнктов. Т.к.
 все дизъюнкты выполнены, то хотя бы одна буква для дизъюнкта встречалась в
 уже выбранных множествах. Тогда для всех непокрытых букв дизъюнкта $\mathcal{D}$ можно
 взять множество вида $\{\mathcal{D}, \dots\}$. $\Rightarrow$ наши покрытие.

$\Leftarrow$ Пусть есть покрытие. Тогда для каждой переменной в покрытии есть множество
 $\{x, x_{a_1}\}$. $\rightarrow$ можно восстановить набор. Осталось понять, почему набор выполни-
 мый. Т.к. в наборе множеств нет множеств вида: $\{\mathcal{D}, \mathcal{D}_1; \mathcal{D}_2; \mathcal{D}_3\}$, то как-

данный дизъюнкт попал куда-то к переменной, а там везде значение $1 \Rightarrow$ все дизъюнкты выполнялись $\Rightarrow \varphi$ -выполнима

Таким образом; $3SAT \leq_p \text{exactsetcover} \Rightarrow \text{exactsetcover} \in \text{NPC}$ ■

$$(9) \text{SUBSET-SUM} = \{(k_1, k_2, \dots, k_m; N) \mid \exists d \in \{0, 1\}^m : \sum_{i=1}^m d_i k_i = N\}$$

Теорема (8/9) SUBSET-SUM $\in \text{NPC}$

$$(10) \text{KNAPSACK} = \{(w_1, w_2, \dots, w_n); (v_1, v_2, \dots, v_n); W, V \mid \exists x \in \{0, 1\}^n : \sum_{i=1}^n x_i v_i \geq V, \sum_{i=1}^n x_i w_i \leq W\}$$

Комментарий: Вот таким непростым образом написана одна из вариаций задачи о рюкзаке

Теорема (8/9) KNAPSACK $\in \text{NPC}$

Часть 6

Полиномиальная иерархия. Самосводимость. Задача оптимизации.

Аппроксимация.

Опр Язык A принадлежит классу Σ_n^P , если его можно представить в виде $x \in L \Leftrightarrow \exists y_1 \forall y_2 \exists y_3 \dots ? y_n V(x, y_1, y_2, \dots, y_n)$, где $?$ - какой-то квантор, зависит от четности n , $V(x, y_1, y_2, \dots, y_n)$ - ДНТ, работающая за $\text{poly}(|x|)$.

Опр Язык A принадлежит классу Π_n^P , если его можно представить в виде $x \in L \Leftrightarrow \forall y_1 \exists y_2 \forall y_3 \dots ? y_n V(x, y_1, y_2, y_3, \dots, y_n)$, где $?$ - какой-то квантор, зависит от четности n , $V(x, y_1, y_2, y_3, \dots, y_n)$ - ДНТ, работающая за $\text{poly}(|x|)$.

$$\forall n \Sigma_0^P = \Pi_0^P = P, \Sigma_1^P = NP, \Pi_1^P = coNP$$

□ $\Sigma_0^P = \Pi_0^P = P$, часто выбирают как базу, но ещё это вытекает из общего определения, если выбросить все кванторы, то останется $V(x)$ - ДНТ, работающая за полином от $|x|$.

$A \in \Sigma_1^P \Leftrightarrow \exists s V(x, s) = 1$ - совпадает с сертификатным определением NP, аналогично

Π_1^P совпадает с сертификатным определением coNP ■

$$\forall n \Sigma_n^P \subseteq \Sigma_{n+k}^P \cap \Pi_{n+k}^P; \Pi_n^P \subseteq \Sigma_{n+k}^P \cap \Pi_{n+k}^P$$

□ Доказывается аналогично арифметической иерархии дописыванием фиктивных кванторов и переменных, однако про полиномиальную иерархию неизвестно, строгое ли вложение. ■

$$\text{Опр } PH = \bigcup_{k=0}^{\infty} \Sigma_k^P$$

$$\forall n \Pi_n^P \subseteq PH$$

□ δ/g ■

Теорема Эквивалентны следующие 3 утверждения:

1) $\Sigma_k^P = \Pi_k^P$

2) $\Sigma_k^P = \Sigma_{k+1}^P$

3) $PH = \Sigma_k^P$

$\square (1 \Rightarrow 2)$ Б.о.о $k \geq 2$, тогда выберем произвольный язык $B \in \Sigma_{k+1}^P$, и он будет записываться в виде $\exists y_1 \forall y_2 \dots \forall y_{k+1} V(x, y_1, \dots, y_{k+1})$. Тогда язык пар $(x, y_1) \in \Pi_k^P$, т.к. записывается в виде $\forall y_2 \exists y_3 \dots \forall y_{k+1} V(x, y_1, y_2, \dots, y_{k+1})$. По предположению $\Sigma_k^P = \Pi_k^P \Rightarrow$

$\Rightarrow$ язык пар $(x, y_1) \in \Sigma_k^P \Rightarrow \exists$ формула $\exists z_2 \forall z_3 \dots \exists z_{k+1} \varphi(x, y_1, z_2, \dots, z_{k+1})$, где φ - верификатор.

тор. $\Rightarrow$ язык $B \in \Sigma_{k+1}^P$ представим в виде $\Sigma_k^P \Rightarrow \Sigma_k^P = \Sigma_{k+1}^P$.

$(2 \Rightarrow 1)$ Пусть $\Sigma_k^P = \Sigma_{k+1}^P$. Т.к. $\Pi_k^P \subseteq \Sigma_{k+1}^P$, то $\Pi_k^P \subseteq \Sigma_k^P$. Р-м $B \in \Sigma_k^P$. Тогда $\overline{B} \in \Pi_k^P \Rightarrow \overline{B} \in \Sigma_k^P \Rightarrow B \in \Pi_k^P \Rightarrow \Sigma_k^P \subseteq \Pi_k^P \Rightarrow \Sigma_k^P = \Pi_k^P$.

$(1 \Leftrightarrow 2) \Rightarrow 3$ Пусть $\Sigma_k^P = \Pi_k^P = \Sigma_{k+1}^P$. Хотим доказать, что $\forall m \in \mathbb{N} \hookrightarrow \Sigma_{k+m}^P = \Sigma_k^P$. Тогда будет выполнено, что $PH = \bigcup_{k=1}^{\infty} \Sigma_k^P = \bigcup_{k=1}^{\infty} \Sigma_n^P = \Sigma_k^P$ в силу вложенности. Докажем по индукции.

База: $m=1$ - утверждение 2.

Сразу скажем, что в таком случае, если $\Sigma_k^P = \Pi_k^P = \Sigma_{k+m}^P \Rightarrow \Pi_{k+m}^P = \Sigma_{k+m}^P$.

Пусть верно для всех $m \leq n$. Докажем, что верно для $m=n+1$. Тогда нам известно, что $\Sigma_{k+n}^P = \Pi_{k+n}^P$, а надо доказать, что $\Sigma_{k+n+1}^P = \Sigma_{k+n}^P$, но мы ровно то же самое доказывали при $1 \Rightarrow 2$, т.е. можно повторить рассуждения $\Rightarrow PH = \Sigma_k^P$.

$(3 \Rightarrow 2)$ Пусть $PH = \Sigma_k^P$. Тогда $\Sigma_{k+1}^P \subseteq PH$, но $\Sigma_k^P \subseteq \Sigma_{k+1}^P \cap PH \Rightarrow \Sigma_k^P = \Sigma_{k+1}^P$ ■

Утв (δ/g) В Σ_k^P и P_k^P есть Σ_k^P -полные и P_k^P -полные задачи соответственно.

Опр Σ_k^P -SAT = $\{ \varphi \mid \exists y_1 \forall y_2 \dots \exists y_k \varphi(y_1, y_2, \dots, y_k) = 1 \}$, где y_i - группа переменных

Нетрудно заметить, что Σ_1^P -SAT = SAT = $\{ \varphi \mid \exists y \varphi(y) = 1 \}$, где y - набор значений переменных

Утв (δ/g) Σ_k^P -SAT $\in \Sigma_k^P C$ (Σ_k^P -полная задача); P_k^P -SAT $\in P_k^P C$

Утв Пусть $L \in NPC$. Пусть у нас есть оракул, который может за $O(1)$ получить ответ на вопрос $x \in L$. Тогда $\exists$ полиномиальный алгоритм, решающий задачу поиска для L . Это называется самосводимостью.

□ Точного доказательства не было, только идея:

1) Сводим A к 3-SAT

2) Решаем задачу поиска для 3-SAT. Это делается очень просто. Пусть $\varphi \in 3\text{-SAT}$.

Выберем произвольную переменную x и подставим $x=0$. Спросим у оракула для

3-SAT, выполняется ли формула с подстановкой $x=0$. Если да, то ставим $x=0$, иначе $x=1$.
учитывая предыдущие результаты

Продолжив такое для всех переменных, получаем пример, то есть решили задачу поиска.

3) Вести результат обратно ■

Опр Задача оптимизации - такая задача, что для каждого ее экземпляра x мы имеем множество решений $F(x)$ и для каждого решения $s \in F(x)$ определена функция стоимости $c(s)$, являющаяся натуральным числом. Тогда

оптимальная стоимость решения определяется как $OPT(x) = \max_{S \in F(x)} c(S)$ (или $\min$ в зависимости от задачи)

Пример задачи: Рассмотрим $CLIQUE$. Пусть x - набор вершин графа G (исходного), являющийся кликой, $c(x)$ - количество вершин в x . Определим $OPT(x) = \max_{S \in F(x)} c(S)$,

то есть получим задачу поиска максимальной клики.

Опр Пусть есть алгоритм M , который по экземпляру задачи x возвращает решение $M(x) \in F(x)$. Тогда мы будем говорить, что M - алгоритм ϵ -аппроксимации ($\epsilon \in [0, 1]$),

$$\text{если } \forall x \hookrightarrow \frac{|c(M(x)) - OPT(x)|}{\max(c(M(x)), OPT(x))} \leq \epsilon$$

Опр Порог аппроксимации A - точная нижняя грань всех таких ϵ , что существует алгоритм ϵ -аппроксимации для A , работающий за полиномиальное время.

Пример: Приведём алгоритм ϵ -аппроксимации для поиска вершинного покрытия с $\epsilon = \frac{1}{2}$. Надо минимизировать вершинное покрытие, поэтому в данном алгоритме требуется найти вершинное покрытие S такое, что $\frac{||S| - a|}{\max(|S|, a)} \neq \frac{1}{2}$, то есть $|S|$ превосходит правильный ответ a не более, чем в 2 раза.

Алгоритм такой:

- 1) Заведём множество вершин S и множество рёбер Q , изначально пустые.
- 2) Проходим по всем рёбрам графа. Пусть мы рассматриваем ребро (u, v) . Если $u \notin S \wedge v \notin S$, то добавляем u и v в S , а в Q добавляем (u, v) .

Тогда на выходе получаем, что S - вершинное покрытие, и $|S| = 2/|Q|$.

A и Q лежат ребра без одинаковых вершин, т.е. паросочетание. Тогда

$$a \geq |Q| \Rightarrow a \geq \frac{|S|}{2} \Leftrightarrow \frac{a}{|S|} \geq \frac{1}{2}. (*)$$

Тогда оценим, насколько $|S|$ отличается от правильного ответа. Заметим, что

$|S| \geq a$. Тогда:

$$\frac{|S| - a}{\max(|S|, a)} = \frac{|S| - a}{|S|} = 1 - \frac{a}{|S|} \leq \frac{1}{2} \text{ в силу } (*) \Rightarrow \text{составили}$$

алгоритм $\frac{1}{2}$ -аппроксимации вершинного покрытия.

Ув(б/г) Если $P \neq NP$, то для задачи коммивояжера нет ^{полиномиальной} ϵ -аппроксимации для $\forall \epsilon < 1$, то есть порога аппроксимации равен 1.

Опр Полиномиальная аппроксимационная схема для задачи оптимизации A - такой алгоритм, что для $\forall \epsilon > 0$ и экземпляра задачи x он является алгоритмом ϵ -аппроксимации и работает за полином от $|x|$. (полином может зависеть от ϵ).

Опр Полностью полиномиальная аппроксимационная схема - полиномиальная аппроксимационная схема, которая работает и за полином от $\frac{1}{\epsilon}$.

Часть 7

Вероятностные вычисления. Класс BPP. Амплификация

Опр Вероятностная МТ - ЭМТ M с двумя аргументами:

- x - сам аргумент для входа
- r - случайные биты

Опр Результат работы $M(x; r)$ - вероятностное распределение, индуцированное конкретным выбором x и равномерным распределением на всех значениях r .

Опр Время работы машины M на данном x - максимальное время работы $M(x; r)$ на всех r указанной длины.

Эквивалентные модификации:

- 1) У МТ есть специальная (бесконечная) лента только для чтения, полностью заполненная независимыми равномерно распределёнными случайными битами. Время работы считается как функция от длины основного входа x .
- 2) У МТ есть 2 функции перехода, каждый раз машина выбирает случайную из них равномерно и независимо от предыдущих выборов.
- 3) У МТ может быть специальное состояние, после прихода в которое машина получает новый случайный бит (равновероятно и независимо от предыстории).

Опр ЭМТ M называется полиномиальной вероятностной машиной Тьюринга, если $\exists$ машина P такой, что на $\forall$ входе $x \in \{0, 1\}^*$ и $\forall r \in \{0, 1\}^{p(|x|)}$ машина M останавливается за $\leq p(n)$ шагов.

Мы считаем ответом машины случайную величину $M(x; U(2^{P(n)}))$, где $U(m)$ обозначает дискретное равномерное распределение на m значениях.

Зам Поскольку результатом работы является случайная величина, то для определения различных классов необходимо оценивать вероятности ответов и ошибок при этих ответах. Поэтому будет удобно задавать классы табличками.

Опр Классом BPP (bounded-error probabilistic polynomial) называется класс языков A , для которых $\exists$ полиномиальный в худшем случае вероятностный алгоритм V такой, что

	$P(V(x;r)=1)$	$P(V(x;r)=0)$
$x \in A$	$\geq \frac{2}{3}$	$\leq \frac{1}{3}$
$x \notin A$	$\leq \frac{1}{3}$	$\geq \frac{2}{3}$

Утв Вероятность ошибки, оцениваемая $\frac{1}{3}$, выбрана случайно, на самом деле можно выбрать $\forall r < \frac{1}{2}$ и получить $\frac{1}{2}$.

□ Докажем чуть далее амплификацией ■

Опр Амплификация алгоритма - уменьшение вероятности путём увеличения времени работы.

Амплифицируем класс BPP:

Пусть есть МТ M , распознающая $L \in \text{BPP}$ с вероятностью ошибки p за время $O(n^k)$. Запустим M n раз, и в качестве ответа возьмём результат большинства запусков. Оценим вероятность ошибки в таком случае.

У нас есть n независимых испытаний Бернулли, т.е. ДСВ Z с биномиальным распределением, хотим найти вероятность того, что у нас $\geq \frac{n}{2}$ ошибок

По биному Ньютона:

$$1 = \sum_{k=0}^n C_n^k p^k (1-p)^{n-k} = \underbrace{\sum_{k=0}^{\frac{n}{2}-1} C_n^k p^k (1-p)^{n-k}}_{\text{Здесь } < \frac{n}{2} \text{ ошибок, т.е. не подходит}} + \underbrace{\sum_{k=\frac{n}{2}}^n C_n^k p^k (1-p)^{n-k}}_{q - \text{искомая иловая вероятность ошибки, которую надо оценить}}$$

$$q = \sum_{k=\frac{n}{2}}^n C_n^k p^k (1-p)^{n-k} = \sum_{t=0}^{\frac{n}{2}} C_n^{\frac{n}{2}+t} p^{\frac{n}{2}+t} (1-p)^{\frac{n}{2}-t} = (p(1-p))^{\frac{n}{2}} \sum_{t=0}^{\frac{n}{2}} C_n^{\frac{n}{2}+t} p^t (1-p)^{-t} \leq$$

$$\leq (\sqrt{p(1-p)})^n \cdot 2^{\frac{n}{2}} = (2\sqrt{p(1-p)})^{\frac{n}{2}}$$

$$\sqrt{p(1-p)} < \frac{p+(1-p)}{2} = \frac{1}{2} \quad \text{при } p \neq 1-p \text{ (среднее геометрическое и среднее арифметическое)}$$

$$2\sqrt{p(1-p)} < 1 \Rightarrow (2\sqrt{p(1-p)})^{\frac{n}{2}} < 1$$

Заметим, что мы сделали n запусков M , которая работает $O(n^k)$ времени, т.е. получили $O(n^{k+1})$ -время работы. Если сделать не n , а n^m запусков, то вероятность ошибки $q < (2\sqrt{p(1-p)})^{\frac{n^m}{2}}$, и время работы будет $O(n^{k+m})$

Заметим, что $(2\sqrt{p(1-p)})^{\frac{n^m}{2}} \xrightarrow{m \rightarrow \infty} 0 \Rightarrow$ можно уменьшить вероятность ошибки до $\forall \epsilon \in (0, \frac{1}{2})$, и оставить полиномиальное время работы.

Из этого следует, что число $\frac{1}{3}$ в BPP можно заменить на $\forall \epsilon \in (0, \frac{1}{2})$ и получить тот же класс.

Часть 8

Вероятностные сложности классы. Дерандомизация. Метод условных вероятностей

Опр $\exists d, \beta: \mathbb{N} \rightarrow [0, 1]$ и $\forall n \in \mathbb{N}, 0 \leq d(n) < \beta(n) \leq 1$. Классом $BPP_{d(n), \beta(n)}$ назовём множество таких языков A , для которых $\exists$ полиномиальный в худшем случае вероятностный алгоритм V такой, что:

$$x \in A \Rightarrow P(V(x; r) = 1) \geq \beta(|x|)$$

$$x \notin A \Rightarrow P(V(x; r) = 1) \leq d(|x|)$$

Зам Задавая различные $d(n)$ и $\beta(n)$ можно получить много различных классов (даже NP)

Опр Обозначим через BPP_r аналог BPP , где все вероятности берутся по распределению на Γ такому, что его биты независимы и равны 0 с вероятностью r и 1 с вероятностью $1-r$.

$$\text{Утв } \forall r \in (0, 1) \hookrightarrow BPP \subseteq BPP_r \quad (\delta/g)$$

Утв (δ/g) Если $r \in (0, 1)$ - полиномиально вероятностно вычисляемое число, т.е. i -ый знак его двоичного разложения можно получить вероятностным алгоритмом за полиномиальное от i время, то $BPP_r = BPP$. Для некоторых невысказанных r класс BPP_r шире BPP . Такие r образуют всюду плотное на $(0, 1)$ множество.

Опр Классом RP (probabilistic polynomial-time) называется класс языков A , для которых $\exists$ полиномиальный в худшем случае вероятностный алгоритм V такой, что:

RP	$P(V(x;r)=1)$	$P(V(x;r)=0)$
$x \in A$	$> \frac{1}{2}$	$< \frac{1}{2}$
$x \notin A$	$< \frac{1}{2}$	$> \frac{1}{2}$

Опр Классом RP (randomized polynomial-time) называется класс языков A , для которых $\exists$ полиномиальный в худшем случае вероятностный алгоритм V такой, что:

RP	$P(V(x;r)=1)$	$P(V(x;r)=0)$
$x \in A$	$\geq \frac{1}{2}$	$\leq \frac{1}{2}$
$x \notin A$	0	1

Опр Классом coRP называется класс языков A , для которых $\exists$ полиномиальный в худшем случае вероятностный алгоритм V такой, что:

coRP	$P(V(x;r)=1)$	$P(V(x;r)=0)$
$x \in A$	1	0
$x \notin A$	$\leq \frac{1}{2}$	$\geq \frac{1}{2}$

Важные проблемы вероятностных алгоритмов:

- Могут с некоторой вероятностью ошибаться
- При реализации на практике часто используют генератор псевдослучайных чисел, это может спровоцировать отклонение их поведения от теоретического.

Случайность можно рассматривать как ресурс, как память и время.

Опр Дерандомизация - это процесс удаления случайности (или ее минимального использования).

Естественным способом дерандомизации является нахождение эффективного метода поиска в соответствующем пространстве событий Ω точки ω , хорошей относительно входа x . Точка ω называется хорошей относительно x , если $A(x; \omega) = f(x)$. Проблема состоит в том, что пространство событий, как правило, имеет экспоненциальный размер.

Методы дерандомизации:

- 1) Метод условных вероятностей
 - 2) Метод малых пространств событий (не будем рассматривать)
-

Метод условных вероятностей:

- На каждом шаге текущее пространство событий делится на две равные по размеру части, для каждой из которых вычисляется условная вероятность нахождения в ней хорошей точки.
- После этого поиск продолжается в половине, для которой посчитанная вероятность оказалась выше.
- Поиск завершается через $\log_2 |\Omega|$ шагов, когда в пространстве окажется всего одна точка (которая должна быть хорошей).
- Метод работает, если такие условные вероятности могут быть эффективно посчитаны (или хотя бы приближены).

Рассмотрим пространство событий $\Omega = \{0,1\}^n$. $\{A_1, A_2, \dots, A_m\}$ - набор событий, а $X_1, X_2, \dots, X_m$ - являются их характеристическими переменными (т.е. $P(X_i = 1) = p_i$;

$P(X_i = 0) = 1 - p_i$; $\forall i \in \overline{1, m}$). Рассмотрим случайную величину $X = \sum_{k=1}^m X_k$.

$$\text{Тогда } E(X) = E\left(\sum_{k=1}^m X_k\right) = \sum_{k=1}^m E(X_k) = \sum_{k=1}^m (1 \cdot p_k + 0 \cdot (1 - p_k)) = \sum_{k=1}^m p_k.$$

$E(X) \geq k \Leftrightarrow$ Есть точка $(a_1, a_2, \dots, a_n)$ в пространстве событий, в которой выполняются хотя бы k событий.

Введём n случайных переменных: $y_1, y_2, \dots, y_n$, где $\forall i \in \overline{1, n}$ и y_i принимает значение 0 или 1 независимо и равновероятно. Мы находим биты $a_1, a_2, \dots, a_n$. Пусть $a_1, a_2, \dots, a_{j-1}$ уже выбраны. Выбираем значение для a_j в зависимости от значения

условного математического ожидания:

$$E(X | a_1, a_2, \dots, a_j) = \sum_{k=1}^m P(A_k | a_1, a_2, \dots, a_j), \text{ где под } a_1, a_2, \dots, a_j \text{ понимается событие}$$

$$y_1 = a_1 \wedge y_2 = a_2 \wedge \dots \wedge y_j = a_j.$$

Если выбирать значения a_{j+1} так, чтобы максимизировать значение $E(X | a_1, a_2, \dots, a_{j+1})$,

то его значение не будет уменьшаться. Поскольку в начале оно было не менее k , то

оно будет $\geq k$ и в конце. Но в конце у каждого a_i уже есть значение $\Rightarrow$

$E(X | a_1, a_2, \dots, a_n)$ будет равно количеству событий, выполненных в данной точке.

Пример:

Возьмём задачу 3-SAT. Заметим, что $P(a \vee b \vee c = 1) = \frac{7}{8}$. Пусть ДСВ X - количество

выполненных дизъюнктов. и в формуле φ n дизъюнктов. Тогда:

$$E(X) = \sum_{k=1}^n E(X_k) = \frac{7}{8} n.$$

То есть мы хотим алгоритм, который ищет набор, на котором выполнено хотя бы $\frac{7}{8}n$ дизъюнктов.

$$E(x) = E(x|x_1=1)P(x_1=1) + E(x|x_1=0)P(x_1=0) = \frac{1}{2}E(x|x_1=1) + \frac{1}{2}E(x|x_1=0)$$

Тогда хотя бы одно из чисел $E(x|x_1=0)$ и $E(x|x_1=1) \geq E(x) = \frac{7}{8}n$.

Алгоритм такой:

1) Если $E(x|x_1=0) \geq \frac{7}{8}n$, то $x_1=0 \rightarrow$ оператор присваивания. Пусть $x_1=d_1$
Иначе $x_1=1$

2) Берём переменную x_2 . Считаем $E(x|x_2=1 \wedge x_1=d_1)$ и $E(x|x_2=0 \wedge x_1=d_1)$ и выбираем большее

3) Повторяем m раз, где m - количество переменных

↓
В результате из вероятностного получим полиномиальный детерминированный алгоритм, который находит набор, на котором выполнено хотя бы $\frac{7}{8}n$ дизъюнктов.

Часть 9

Сложность по памяти. Классы $DSPACE$, $NSPACE$, $PSPACE$, $NPSPACE$. Теорема Сэвиги.

Опр Функция $s: \mathbb{N} \rightarrow \mathbb{N}$ называется конструируемой по памяти, если $\exists$ МТ, которая по входу 1^n вычисляет $s(n)$, используя $O(s(n))$ памяти.

Опр Ячейка памяти К работей ленты МТ М была использована, если считывающее устройство останавливалось на ней хотя бы один раз. Таким образом, затраженная память — количество использованных ячеек всех рабочих лент МТ М.

Опр $\exists s(n)$ — неубывающая функция. Классом $DSPACE(s(n))$ называется класс языков, которые можно распознать на ДМТ, которая на $\forall$ входе длины n использует не более $O(s(n))$ ячеек на рабочих лентах (то есть использует не более $O(s(n))$ памяти).

Опр $\exists s(n)$ — неубывающая функция. Классом $NSPACE(s(n))$ называется класс языков, которые можно распознать на НМТ, на $\forall$ входе длины n использующей не более $O(s(n))$ памяти.

Теорема (Сэвиги). Если $s(n) \geq \log n$, то $NSPACE(s(n)) \subseteq DSPACE(s^2(n))$

$\square \exists L \in NSPACE(s(n)) \Rightarrow \exists$ НМТ, распознающая L на памяти не более $s(|x|)$ для

$\forall x$. То есть каждая ветка конфигураций использует максимум $s(|x|)$ памяти. Пусть у

НМТ 2 ленты: входная и рабочая. Входную ленту нельзя менять, можно только читать $\Rightarrow$

мы это не считаем за затраченную память, потому что всё идёт дополнительная

память. Пусть слово x дано на входной ленте. Тогда на ней всего $n = |x|$

содержательных конфигураций (меняется только положение считывающего устройства).

На рабочей ленте используется $\leq s(n)$ памяти $\Rightarrow$ длину конфигурации можно урезать до $s(n)$ эл-ов. Тогда всего конфигураций на рабочей ленте:

$$\underbrace{|\Sigma|}_{\substack{\text{на каждой ячейке} \\ \text{любой символ}}} \cdot \underbrace{s(n) \cdot |Q|}_{\substack{\text{перебираем положение устройства +} \\ \text{символ состояния}}} \cdot \underbrace{|\Sigma|^{s(n)}}_{\substack{\text{длина конфигурации} \\ \text{на рабочей ленте}}}$$

на каждой ячейке любой символ

перебираем положение устройства + символ состояния.

Тогда у всей НТ $n s(n) |Q| |\Sigma|^{s(n)}$ конфигураций. Заметим, что при $s(n) \rightarrow \log n$ и $n s(n) |Q| |\Sigma|^{s(n)} = 2^{O(s(n))}$, при этом память на запись одной конфигурации - $O(s(n))$.

Теперь мы будем строить ДНТ на основе данной НМТ, которая распознаёт тот же язык.

Наш алгоритм будет проверять, есть ли в НМТ путь по графу конфигураций от начального состояния и до прикимающего.

Положим в ДНТ код НМТ - константная память.

Определим функцию: $R(u; v; k) = 1 \Leftrightarrow$ из конфигурации u в конфигурацию v можно перейти не более чем за 2^k шагов.

Нам нужно считать $R(u; v; 0)$. Это делается по таблице переходов - просто смотрим, можно ли за 1 шаг добраться до того состояния. Это требует $O(s(n))$ ^{памяти}, т.к. надо лишь записать 2 конфигурации.

Мы посчитали, что у НМТ $2^{O(s(n))} = 2^N$ конфигураций $\Rightarrow$ если принимает слово, то не более чем за такое время. А ещё наложим ограничение на НМТ - пусть она в

конце оставляет пустую ленту и q_{yes} или q_{no} - состояния в зависимости от того, принято слово или нет. Тогда начальную конфигурацию u мы знаем, конечную v , которую хотим - тоже (q_{yes} + пустая лента) и $k=N \Rightarrow$ надо вычислить $R(u, v; N)$

Заметим, что $R(a, b; k) = 1 \Leftrightarrow \exists c: R(a, c; k-1) = 1 \wedge R(c, b; k-1) = 1 \Rightarrow$ для каждого k можно делать перебор по c и уходить от k к $k-1 \Rightarrow$ получаем рекурсивный алгоритм, каждая ветка рекурсии глубины $\leq N = O(s(n))$. Каждый шаг мы сохраняем текущую конфигурацию $\Rightarrow$ всего $O(s^2(n))$ памяти $\Rightarrow$ построим ДМТ, μ распознающую L за $O(s^2(n))$ памяти $\Rightarrow NSPACE(s(n)) \subseteq DSPACE(s^2(n))$ ■

$$\text{Опр } PSPACE = \bigcup_{c=1}^{\infty} DSPACE(n^c) = DSPACE(poly(n))$$

$$\text{Опр } NPSPACE = \bigcup_{c=1}^{\infty} NSPACE(n^c) = NSPACE(poly(n))$$

$$\text{Утв } PSPACE = NPSPACE$$

□ $PSPACE \subseteq NPSPACE$ - т.к. ДМТ является НМТ

$NPSPACE \subseteq PSPACE$ - следует из теоремы Сэвижа ■

$$\text{Утв } NP \subseteq PSPACE$$

□ Т.к. $PSPACE = NPSPACE \Rightarrow$ надо доказать, что $NP \subseteq NSPACE$. Если НМТ работала

время $p(|x|)$, то она не могла посетить $> p(|x|)$ ячеек памяти $\Rightarrow$ работала не более

$p(|x|)$ на памяти ■

Часть 10

PSPACE-трудность и полнота. Примеры PSPACE-полных задач.

Опр Язык B называется PSPACE-трудным, если $\forall A \in \text{PSPACE} \cup A \leq_p B$

Зам Важно обратить внимание, что $\leq_p$ - сводимость по Карпу (т.е. полиномиальная сводимость по времени, а не по памяти).

Опр Язык B называется PSPACE-полным (обозначение $B \in \text{PSPACE}$), если $B \in \text{PSPACE}$ и B - PSPACE-трудный.

Примеры PSPACE-полных задач:

① $\text{SPACE-TSAT} = \{ \langle M, x, 1^s \rangle \mid M(x) = 1 \text{ и } M(x) \text{ занимает не более } s \text{ ячеек памяти} \}$

Теорема $\text{SPACE-TSAT} \in \text{PSPACE}$

□ 1) Сначала докажем, что $\text{SPACE-TSAT} \in \text{PSPACE}$. Нужно построить ЭМТ A , которая использует полиномиальную память и распознаёт этот язык. A будет моделировать работу M на слове x . Что надо хранить? Надо выделить s ячеек свободной памяти для работы на них M и ещё будем считать шаги. Мы уже доказывали, что если МТ использует $O(\text{poly}(x))$ ячеек памяти, то её время работы можно ограничить $2^{O(\text{poly}(s))}$ шагами, т.к. количество конфигураций такое. Для записи числа шагов в двоичной системе требуется $O(\text{poly}(\log(s)))$ памяти $\Rightarrow$ полное моделирование A тратит $O(\text{poly}(\log(s)))$ памяти $\Rightarrow$ построили необходимую ЭМТ $\Rightarrow \text{SPACE-TSAT} \in \text{PSPACE}$

2) Теперь докажем, что SPACE-TSAT - PSPACE-трудный.

Выберем произвольный $L \in \text{PSPACE}$. Для него $\exists M_L$ - ЭМТ, которая тратит не более

$p(x)$ ячеек памяти. Наша функция f по слову x будет выдавать тройку $(M_L; x; p(x))$. Из построения Π , что $x \in L \Leftrightarrow (M_L; x; p(x)) \in \text{SPACE}^{\text{MSAT}}$.

Осталось доказать, что f вычислима за полиномиальное время. Выписывание M_L занимает $O(1)$ времени, т.к. не зависит от x . Выписывание x занимает $O(|x|)$ времени. Подсчет и запись $p(x)$ в унарной записи займёт $O(p(x))$ времени $\Rightarrow f$ вычисляется за полиномиальное время $\Rightarrow \forall L \in \text{PSPACE} \text{ и } L \leq_p \text{SPACE}^{\text{MSAT}} \Rightarrow \text{SPACE}^{\text{MSAT}} - \text{PSPACE-трудный}$.

т.к. $\text{SPACE}^{\text{MSAT}} \in \text{PSPACE}$ и он P-SPACE-трудный $\Rightarrow \text{SPACE}^{\text{MSAT}} \in \text{PSPACE} \text{ с } \blacksquare$

② $\text{SUCCINCTPATH} = \{(\varphi; u; v) \mid \text{в графе, построенном по формуле } \varphi \text{ есть путь из } u \text{ в } v\}$

Для начала поймаем формат входных данных. Для этого надо разобраться как граф задавать формулой. В графе G N вершин. Пронумеруем их, чтобы смог записать все вершины бинарным кодом, требуется $k = \lceil \log_2 N \rceil$ символов. Что же делает формула φ ? По заданным двум вершинам u и v в двоичной системе она выдаёт 1 тогда и только тогда, когда есть ребро между u и v . Т.е. $\varphi(u; v) = 1 \Leftrightarrow \text{в } G \text{ есть ребро между } u \text{ и } v$. Т.к. $u = u_1 u_2 \dots u_k$; $v = v_1 v_2 \dots v_k$, то φ - булева формула от $2k$ переменных $\exists |\varphi| = p(k)$, где p - полином. Таким образом входные данные полиномиальны от k .

Теорема $\text{SUCCINCTPATH} \in \text{PSPACE}$

□ 1) Сначала докажем, что $\text{SUCCINCTPATH} \in \text{PSPACE}$. То есть нам надо найти

проверить, есть ли путь между двумя вершинами в графе. Быстрее всего это делается с помощью dfs, то есть мы буквально его запустим на этом графе, осталось посчитать, за сколько он работает

Так как $|V| = N = \sum_{k=0}^{\infty} 2^k$, то $|E| = 2^{2k} \Rightarrow$ dfs работает за $O(|V| + |E|)$ времени $\Rightarrow O(2^k)$ времени. При этом нам надо использовать полиномиальную от k память.

Вспомним, что $PSPACE = NPSPACE$ и упростим себе жизнь, запровав это на НМТ.

То есть приходя в вершину, следующую мы будем выбирать недетерминированным образом. Заметим, что если есть путь из u в v , то его длина не более $|V|-1$, т.е. $2^k - 1$. Тогда мы можем перебрать все пути длиной $\leq 2^k - 1$. Будем каждый отрезок

пути обрабатывать в своей ветке. Что тогда хранится в ветке? Надо хранить счётчик (cnt), в который мы будем записывать число пройденных вершин $\rightarrow O(k)$

памяти. Пусть мы находимся в некой вершине x и выбираем следующую y .

Тогда если $y = v$, то выводим 1, если $cnt = 2^k - 1$, а мы всё ещё не вышли, то пути в этой ветке нет $\Rightarrow$ возвращаем 0. После каждого выбора новой вершины

++cnt. Итого мы тратим $O(k)$ памяти на cnt, $O(k)$ на хранение нескольких нужных вершин, например, нам хватит x, y и v и $O(p(k))$ на вычисление

$\varphi(x, y)$ каждый раз $\Rightarrow$ каждая ветка НМТ работает за $O(p(k))$ по памяти $\Rightarrow$

$\Rightarrow$ $SUCCINCTPATH \in NPSPACE \Rightarrow SUCCINCTPATH \in PSPACE$

2) Теперь докажем, что $SUCCINCTPATH$ - $PSPACE$ -трудный. Выберем произвольный

$L \in PSPACE$. Тогда $\exists M_L$, которая решает $x \in L$ за $p(|x|)$ по памяти.

Тогда каждая конфигурация задаётся длиной $m \cdot \log(|\Sigma|)$, где m - константа, необходимая для перекодирования в бинарную запись. Таблица δ NT M , можно считать заданной формулой для графа конфигураций, потому что мы можем перебирать символы и переходить по символу к новой конфигурации. Тогда δ по входу x выдаст M , начальную конфигурацию N на слове x и конечную принимающую конфигурацию (можно сделать из пустой ленты и состояния q_{acc}). Тогда задача $x \in L$ эквивалентна тому, что есть путь от начальной конфигурации до принимающей в графе конфигураций. Осталось показать, почему δ полиномиально вычислима по времени. Потому что он передаёт константную NT, и переписывает две конфигурации, которые не надо вычитать. $\Rightarrow$ $\text{SUCCINCTPATH} \in \text{PSPACE}$ - трудный.

т.к. $\text{SUCCINCTPATH} \in \text{PSPACE}$ и $\text{SUCCINCTPATH} \in \text{PSPACE}$ - трудный $\Rightarrow \text{SUCCINCTPATH} \in \text{PSPACE}$ ■

③ $GG = E(G, x)$ | в обобщённой игре города в графе G и начальной вершине x выигрывает 1 игрок }

Сначала надо понять формулировку. У нас есть конечное число городов - вершин графа

G . Если между вершинами u и v есть ребро, то после "города u " можно сказать

"город v ". Спрашивается, есть ли гарантированно выигрышная стратегия для первого

игрока. Выиграть - значит прийти в вершину, из которой нет ребра ни в одну из

несоседних вершин. Гарантированно выигрышная стратегия - значит, что при любом

ходе второго игрока, найдётся следующий ход, который далее приводит в победную

конфигурацию.

Теорема $GG \in PSPACE$

□ δ/g ■

④ $TQBF = \{ \varphi \mid \varphi - \text{выполнимая булева формула с кванторами} \}$

Теорема $TQBF \in PSPACE$

□ Доказательство нестрогое, но идея верная.

Заметим, что для языка GG мы можем написать условие принадлежности к языку, как

$\exists u_1 \forall u_2 \exists u_3 \dots \exists u_n: A(u_1; u_2 \dots u_n)$, где $A(u_1; u_2 \dots u_n)$ проверяет, что между u_i и

u_{i+1} есть ребро и n - число вершин. То есть мы записали предикатный язык в виде

булевой формулы с кванторами $\Rightarrow$ т.к. $GG \in PSPACE$, то $TQBF \in PSPACE$ ■

Часть II

Вычисления на логарифмической памяти. Класс L и NL . Теорема Рейнгольда.

Логарифмическая сводимость. Примеры NL -полных языков. Теорема Иммермана-Селепенки.

Зам Сейчас важно, что теперь у МТ будут отдельно входные ленты (с которых можно только читать), рабочие ленты и выходные (на которых можно писать слева направо и ничего более). Затраченную память мы далее будем считать только на рабочих лентах, то есть можно будет читать слова любой длины с входных лент и писать произвольно на выходные, если не наложим дополнительных условий, и это не будет влиять на использующую память.

$$\text{Опр } L = PSPACE(\log n)$$

$$\text{Опр } NL = NSPACE(\log n)$$

Теорема (Рейнгольда, δ/g) $\square$ $HPATH = \{G; s; t\} \mid \text{в неориентированном графе } G \text{ есть путь из } s \text{ в } t\} \in L$.

Примеры языков класса L :

$$\textcircled{1} LE = \{(x; y) \mid x \leq y\}$$

$\square$ Будем хранить 2 числа - номер текущего символа чисел $x; y$. Брать соответствующие и сравнивать. Каждый счётчик в двоичной системе займёт $\log_2 \lceil \max\{x, y\} \rceil$ памяти $\Rightarrow$
 $\Rightarrow LE \in L \blacksquare$

$$\textcircled{2} ADD_E = \{(x; y; z) \mid x + y = z\}$$

$\square$ Аналогично храним 2 счётчика + сохраняем, есть ли 1 к следующему разряду и

сравниваем с x , то есть просто складываем столбиком ■

③ $MULT = \{ (x, y, z) \mid xy = z \}$

□ Реализуем умножение столбиком, но тут чуть больше надо запомнить цифр ■

④ $TREE = \{ G \mid \text{неориентированный граф } G \text{ является деревом} \}$

□ G -дерево $\Leftrightarrow G$ -связный и в нём $n-1$ рёбро, где n - количество вершин.

Можно посчитать сначала количество рёбер. Потом перебирать вершины от 2 до n и смотреть, есть ли путь от 1 до текущей вершины. Это можно сделать за

логарифм доп. памяти по теореме Рейнгольда. $\Rightarrow TREE \in L$ ■

⑤ Язык правильных скобочных последовательностей

□ Для константного числа видов скобок можно модифицировать скобочный баланс, добавив последующие проверки.

Опр Функция f вычислима на логарифмической памяти $\Leftrightarrow$ язык

$D_f = \{ (x, k) \mid |f(x)| \leq k \} \in L$ и $E_f = \{ (x, i) \mid f(x) = i \} \in L$ и $\max k$, при котором $(x, k) \in D_f$ ограничено полиномом от $|x|$.

Утв (б/г) композиция функций, вычисляемых на логарифмической памяти, вычислима на логарифмической памяти.

Опр Язык A логарифмически сводится к языку B , если f вычисляемая на логарифмической памяти функция f такая, что $x \in A \Leftrightarrow f(x) \in B$. Обозначение $A \leq_L B$.

Свойства логарифмической сводимости:

- 1) $\forall A \hookrightarrow A \leq_L A$
- 2) $\forall A, B, C: A \leq_L B; B \leq_L C \Rightarrow A \leq_L C$
- 3) $\forall A \in L; \forall B \in \emptyset \text{ и } B \in \{0, 1\}^* \hookrightarrow A \leq_L B$
- 4) $\forall A, B: B \in L: A \leq_L B \Rightarrow A \in L$
- 5) $\forall A, B: B \in NL: A \leq_L B \Rightarrow A \in NL$

Опр Язык B называется NL -трудным, если $\forall A \in NL \hookrightarrow A \leq_L B$

Опр Язык B называется NL -полным, если $B \in NL$ и B - NL -трудный. Обозначение: $B \in NLC$.

Опр (сертификатное определение NL)

$A \in NL \Leftrightarrow \exists V \vdash MT: V(x; s)$ (то есть принимает 2 аргумента и выполняет условие:

- 1) V - ДМТ, которая работает на логарифмической памяти
- 2) Слово x находится на своей входной ленте, и его можно читать без ограничений.
- 3) Слово s находится на второй входной ленте, и его можно читать всего один раз слева направо.
- 4) $x \in A \Leftrightarrow \exists s V(x; s) = 1$

Примеры NL -полных языков:

① $PATH = \{ \langle G; s; t \rangle \mid \text{в ориентированном графе } G \text{ есть путь из } s \text{ в } t \}$

Теорема (с/г) $PATH \in NLC$

② $SCONNECTED = \{G \mid \text{ориентированный граф } G \text{ сильно связный}\}$.

Теорема $SCONNECTED \in NLC$

□ 1) Сначала докажем, что $SCONNECTED \in NL$. Надо проверить, что есть путь от $\forall$ вершин и до $\forall$ вершин v . Сделаем перебор по парам (u, v) . Т.к. $DPATH \in NL$, то можно за логарифм по памяти на НЛТ проверить, есть ли путь из u до v . Если нет, то выводим 0, если да, то продолжаем перебор. После перебора выводим 1, т.к. иначе бы уже было 0. Итого, мы на каждом шаге цикла используем логарифмическую память, а с каждым новым шагом эта память может быть периспользована $\Rightarrow$

$\Rightarrow SCONNECTED \in NL$

2) Теперь докажем, что $SCONNECTED$ — NL -трудный. Для этого хотим $DPATH \leq_L SCONNECTED$. Надо построить функцию f , принимающую на вход $(G; s; t)$, возвращающую граф G' , вычисляемую на логарифмической памяти такую, что $(G; s; t) \in DPATH \Leftrightarrow G' \in SCONNECTED$. Напомним, что, вычисляя f мы можем выводить сколько угодно по памяти, а мы будем выводить граф. $f(G; s; t)$ делает следующее: соединим t со всеми вершинами, кроме s и все вершины, кроме t , соединим с s . Остальные рёбра — рёбра начального графа. Тогда в этом графе G' есть путь от t до $\forall$ другой вершин. Если в G теперь есть путь из s в t , то из $\forall$ вершин мы можем перейти в s , а из s до t , а оттуда до $\forall$ другой вершин $\Rightarrow G'$ сильно связан. Если же в G нет пути из s в t , то и в G' его не будет, т.к. мы добавили только рёбра, входящие в s и исходящие из t .

Таким образом: $(G; s; t) \in \text{DPATH} \Leftrightarrow f(G; s; t) \in \text{SCONNECTED}$.

Поэтому это вычислимо на логарифмической памяти? Пусть граф G дан в виде последовательности рёбер. Тогда надо перенести G , что не требует доп. памяти, кроме сохранения индекса, где мы сейчас находимся. А далее перебор всех вершин, кроме s и t , требует сохранения константного числа вершин и позиций, и каждый шаг просто выводим нужные новые рёбра, и получаем G' на выходной ленте. $\Rightarrow$

$\Rightarrow \text{DPATH} \leq \text{SCONNECTED} \Rightarrow \text{SCONNECTED}$ - NL-трудный.

Т.к. $\text{SCONNECTED} \in \text{NL}$ и NL-трудный $\Rightarrow \text{SCONNECTED} \in \text{NL}$ ■

③ $\text{CYCLE} = \{G \mid \text{в ориентированном графе } G \text{ есть цикл}\}$

Теорема $\text{CYCLE} \in \text{NL}$

□ 1) Сначала докажем, что $\text{CYCLE} \in \text{NL}$. Решим на логарифмической памяти на НМТ.

Переберём все рёбра $u \rightarrow v$. Проверим, есть ли в G путь от v до u . Это можно сделать на логарифмической памяти на НМТ (задача DPATH). $\Rightarrow \text{CYCLE} \in \text{NL}$

2) Докажем, что CYCLE - NL-трудный. Хотим $\text{DPATH} \leq \text{CYCLE}$. Для этого по

$(G; s; t)$ надо построить G' . G' будем выводить сразу на ленту. $V = \{s; v_1; v_2; \dots; v_n; t\}$.

В графе G' будет $n+2$ слоя, и $(n+1)^2$ рёбер. Обозначим для удобства вершины:

$s; i_1; i_2; \dots; i_n; v_1; i_1; v_1; i_2; \dots; v_1; i_n; v_n; i_1; v_n; i_2; \dots; v_n; i_n; t$] в графе G есть ребро $u \rightarrow v$. Тогда в G'

будут рёбра $u_1 \rightarrow v_2; u_2 \rightarrow v_3; \dots; u_n \rightarrow v_{n+1}$. Так мы строим для всех рёбер, это

не требует доп. памяти, кроме хранения информации, какое мы сейчас рассматриваем

ребро. Просто выводим $n-1$ ребро нового графа. Потом добавляем рёбра

$t_1 \rightarrow t_n; t_2 \rightarrow t_n \dots t_{n-1} \rightarrow t_n$. а ребро $t_n \rightarrow S_1$. Докажем, что в G есть путь от s до $t \Leftrightarrow$ в G' есть цикл. Если в G есть путь от s до t , то от S_1 мы дойдем за k шагов до t_k . Из t_k до t_{n-1} , из t_{n-1} до S_1 - наш цикл.

Если нет пути от s до t , то от S_1 мы не дойдем до какого-либо $t_k \Rightarrow$

не дойдем до t_{n-1} , но каждое ребро гарантированно спускает нас на 1 уровень ниже, но единственное ребро вверх - это $t_{n-1} \rightarrow S_1 \Rightarrow$ цикла не будет $\Rightarrow$ ДРАТИ $\leq_L$ CYCLE $\Rightarrow$ CYCLE - NL-трудный

Т.к. CYCLE $\in$ NL и CYCLE - NL-трудный $\Rightarrow$ CYCLE $\in$ NL

Т.к. CYCLE $\in$ NL и CYCLE - NL-трудный $\Rightarrow$ CYCLE $\in$ NL

Теорема (эквивалентность определений NL)

$NL = NL_{\text{серт}}$, где NL - класс, задаваемый как $NL = NSPACE(\log n)$, а

$NL_{\text{серт}}$ - класс задаваемый сертификатным определением.

□ 1) $NL \subseteq NL_{\text{серт}}$. $\exists$ НМТМ, распознающая язык L на логарифмической памяти. $\Rightarrow$ есть принимающая ветка. Запишем в строку выборы всех недетерминированных переходов. $\exists$ ДМТ $V(x;s)$ примет в качестве s эту строку и будет эмулировать одну конкретную ветку НМТ. M на каждой ветке тратит память не более, чем логарифмическую $\Rightarrow V$ работает на логарифмической памяти.

Если принимающей ветки нет, то и сертификата не будет $\Rightarrow NL \subseteq NL_{\text{серт}}$.

2) $NL_{\text{серт}} \subseteq NL$. $\exists$ Есть $V(x;s)$. Построим НМТ M . Она будет эмулировать работу V , но каждый раз при обращении V к ленте s , будет угадывать недетерминиро-

важный переход к следующему биту. Возможность чтения с единжды гарантирует, что недеривированный выбор происходит столько раз, сколько битов в сертификате, что даёт построить НМТ, если зафиксировать длину сертификата ■

$\overline{\text{PATH}} = \{ (G; s; t) \mid \text{в ориентированном графе } G \text{ нет пути из } s \text{ в } t \}$

Теорема (Иммерман - Селенгейм)

$\overline{\text{PATH}} \in \text{NL}$

□ Нам дан граф G и вершины s и t . Предоставим сертификат, показывающий, что из s в t нет пути. Введём обозначение: $N_i(s)$ - множество вершин на расстоянии $\leq i$ от вершины s . Тогда нас интересует вопрос о принадлежности вершины t множеству $N_n(s)$, где $n = |V| - 1$.

1) Рассмотрим задачу о принадлежности вершины x множеству $N_i(s)$. Такая задача,

2) лежит в NL , т.к. сертификатом является последовательность вершин, в которые нужно переходить. А верификатор на каждом шаге проверяет, что $\exists$ ребро в следующую вершину (храним только вершину в которой находимся). Также на каждом шаге увеличиваем счётчик шагов и проверяем в конце, что он $\leq i$.

3) $|N_i(s)| = m$. Покажем, что задача проверки, что вершина t не принадлежит множеству $N_i(s)$ лежит в NL . Передадим в качестве подсказки множество вершин внутри $N_i(s)$ в возрастающем порядке, а после каждой вершины дополнительно передадим сертификат того, что $v \in N_i(s)$ (из п.1). Тогда алгоритм может

проверить, что каждая из переданных вершин действительно $\in N_i(s)$, а возрастающий порядок даёт проверить, что все вершины различны. Также алгоритм проверяет, что все переданные вершины отличны от t и считает их количество. Если передано ровно m вершин, и все они не совпадают с t , то $t \in N_i(s)$.

3) Покажем, что зная, что $|N_{i-1}(s)| = m$ можно показать, что вершина t не лежит во множестве $N_i(s)$. Делается это похоже на шаг 2: передадим в качестве подсказки все вершины из множества $N_{i-1}(s)$. Тогда для каждой вершины необходимо дополнительно проверить, что ~~не переданы вершины~~ из какой вершины нет ребра в t . Это делается, h , с использованием логарифмической памяти.

4) Покажем, что задача проверки, что $|N_{i+1}(s)| = k$ при известной мощности $|N_i(s)| = m$ тоже лежит в NL . Мы покажем, что если $x \in N_{i+1}(s)$, то $\exists$ доказывающий это сертификат (н.1). И если $x \notin N_{i+1}(s)$, то тоже $\exists$ доказывающий это сертификат (н.3). Тогда передадим в качестве сертификата последовательно сертификаты для всех вершин, показывающих, принадлежат ли они $N_{i+1}(s)$. Тогда алгоритм должен проверить все сертификаты и посчитать число подходящих вершин.

5) Осталось соединить всё вместе и описать сертификат. Этого, он состоит из n уровней. На каждом уровне ^{передается} ~~определяется~~ размер отсечного множества $N_i(s)$, потом он проверяется. Далее передаём размер ~~каждого~~ текущего множества и

сертификат для каждой вершины, принадлежит ли она ему. Внутренние сертификаты описаны в п.1 и п.2. В самом конце передаётся сертификат из п.2, доказывающий, что $t \in N_n(s)$. Заметим, что для всех проверок на текущем шаге i требуется знать только $|N_{i-1}(s)|$, что занимает также логарифмическую память.

Таким образом, $\overline{\text{DPATH}} \in NL$ ■

Теорема $NL = coNL$

□ В силу того, что $\text{DPATH} \in NLC \Rightarrow \overline{\text{DPATH}} \in coNLC$. По теореме Immerman-Szelepcsényi $\overline{\text{DPATH}} \in NL \Rightarrow NL = coNL$ ■

Часть 12

Теоремы об иерархиях по времени и по памяти. Диаграмма вложенности классов.

Теорема (об иерархии по времени, δ/g)

Пусть функции $f, g: \mathbb{N} \rightarrow \mathbb{N}$ и конструктивны по времени, и строго монотонно возрастают. Тогда если $\lim_{n \rightarrow \infty} \frac{f(n) \log(f(n))}{g(n)} = 0$, то $\text{DTIME}(f(n)) \subseteq \text{DTIME}(g(n))$

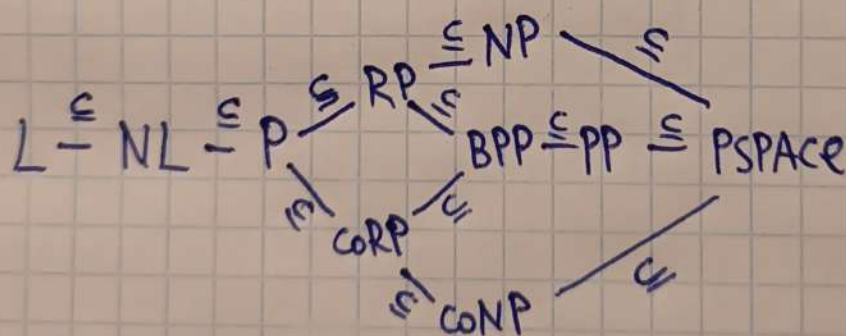
Здесь важно именно, что вложение строгое, нестрогое очевидно.

Теорема (об иерархии по памяти, δ/g)

Пусть функции $f, g: \mathbb{N} \rightarrow \mathbb{N}$, конструктивны по памяти и строго монотонно возрастают.

Тогда если $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$, то $\text{DSPACE}(f(n)) \subseteq \text{DSPACE}(g(n))$

Диаграмма вложенности классов:



В силу того, что $\text{NPSPACE} = \text{coNPSPACE} = \text{DSPACE}$, $\text{coNL} = \text{NL}$, $\text{coBPP} = \text{BPP}$, эти классы не имеет смысла отображать.

Из теоремы об иерархии по памяти следует, что $L \neq \text{PSPACE}$. Строгость всех остальных вложений — всё ещё открытая проблема математики!